

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Recap 1
Coordinate Transforms
Camera Projection
Complementary Filter
Neural Networks

June 15, 2026

Welcome to the lecture!

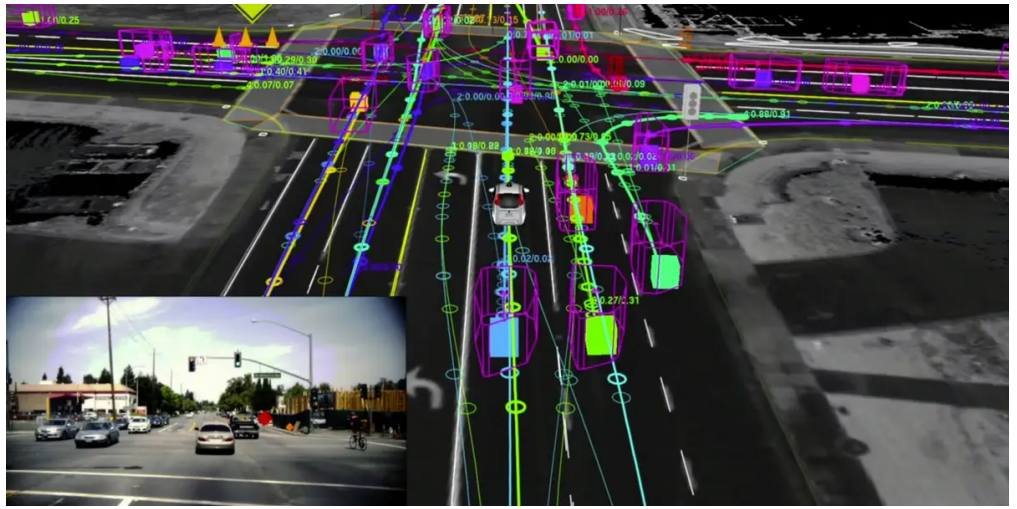
Prof. Dr.-Ing. Björn Giesler
Technische Hochschule Ingolstadt
AIMotion, AI and Sensor Data Fusion
Email: bjoern.giesler@thi.de

- **Preparing for the Exam**
  - Use the slides!!!
    I am updating them to have a bit more consistent variable naming etc., but everything in the exam is in the slides, and in the exercises.
  - Use the exercises!
    I will not have time unfortunately to create more exercise sheets.
    Maybe AI can prepare some sheets for you?
  - I will publish a list of literature, Youtube videos etc. that can also help you prepare.
  - Study together, in pairs or groups!
    There's always someone in a group who has good questions, or good understanding.
- These recap slides are *not enough* to prepare for the exam. Use the original slides and the exercises.

Introduction

Sensor Networks and Sensor Fusion
Motivation

- **End Goal:**
  We Want To Automate A Real-World Task With A Robot.
- **For any automation task operating in the real world, the robot needs a world model that captures the state of the real world accurately enough.**
- **If the task is sufficiently complex, chances are that there is no single sensor able to gather enough data for the world model.**
- **Example for object detection:**
  - Cameras are 2D sensors and cannot measure distance or speed, but are great in azimuth / elevation measurement.
  - Radars cannot determine the class of an object, and have bad azimuth / elevation measurement, but are great in distance measurement.
  - Lidars distort the shape of moving objects and cannot measure speed, but are great in distance and resolution.

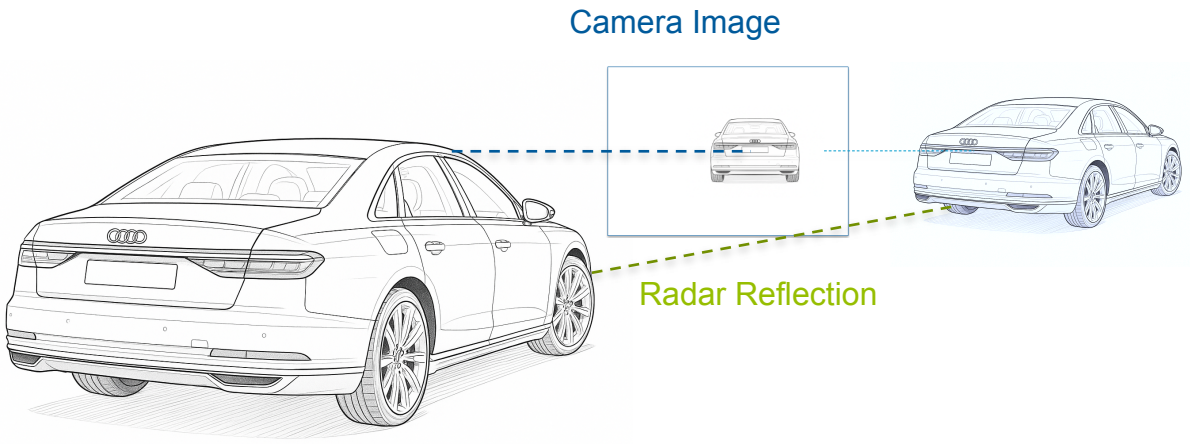


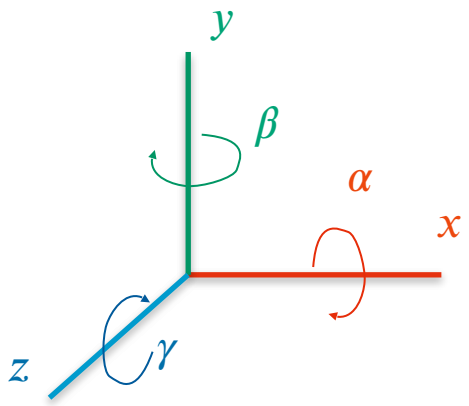




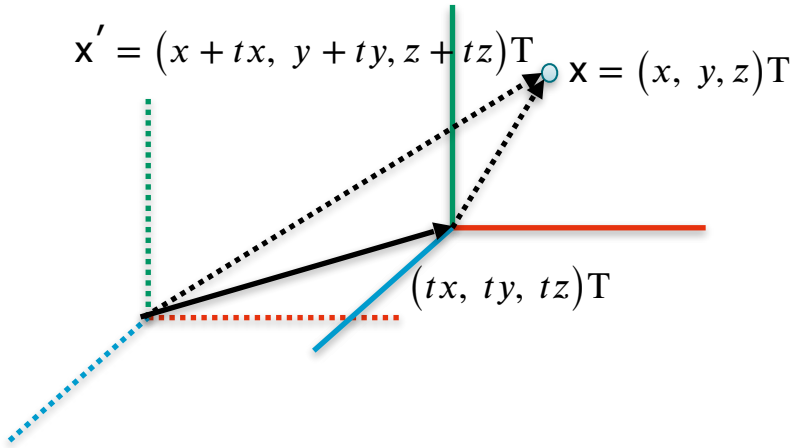
World models are used by self-driving cars and humanoid robots alike.

- **Fusing the output of multiple complementary sensors can generate a sufficient world model.**
- **E.g.:**
  - Detecting a car in a camera video stream (azimuth and elevation)
    $m_{\text{cam}} = (\alpha_{\text{cam}}, \beta_{\text{cam}})$
  - Detecting an object in a radar target list (azimuth and distance)
    $m_{\text{rad}} = (\alpha_{\text{rad}},\ d)$
  - Correlating both if they are the same object
    $\left|\alpha_{\text{cam}} - \alpha_{\text{rad}}\right| < \alpha_{\text{correl}}$
  - Determining 3D coordinates
    x $= \left(d \sin \alpha_{cam}, d \sin \beta_{cam},\ d\right)T$





- We almost always work in right-hand coordinate systems – x is the thumb, y the index, z the middle finger.
- We designate α, β, γ as rotation around the $x,\ y,\ z$ axes respectively.

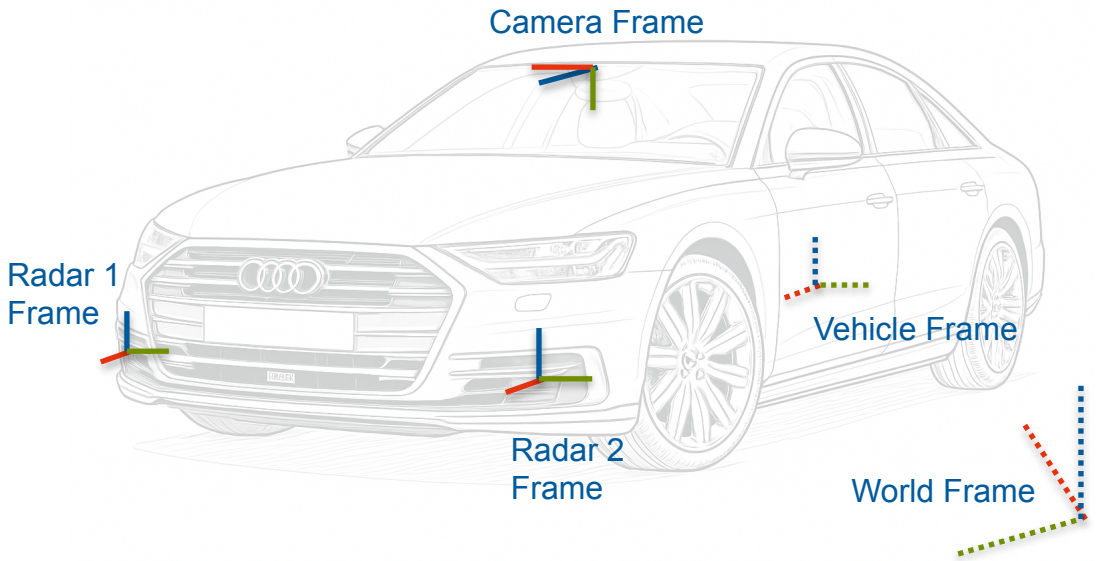


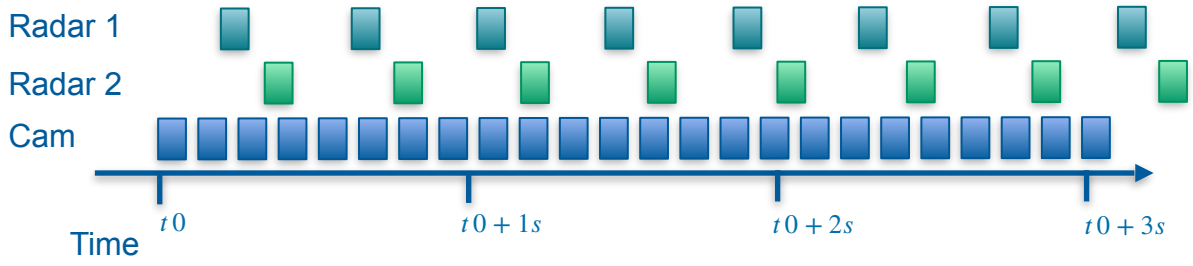
- The same 3D point can be represented in different coordinate systems or **frames.**
- We relate frames with **coordinate transformations.**
- For pure translation (= normal vectors of coordinate systems all parallel), a coordinate transformation is just a vector addition:

  $\mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ and $x' = \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} tx \\ ty \\ tz \end{pmatrix}$ describe the same point in two coordinate frames that are offset by $\begin{pmatrix} tx \\ ty \\ tz \end{pmatrix}$.

Sensor Networks and Sensor Fusion
Interrelating Data from Different Sensors

- **In a multisensor system, sensors are typically geometrically and temporally diverse.**
- **Every sensor provides its output in**
  - its own **coordinate frame**, determined by how and where it is mounted,
  - its own **time frame**, determined by its measuring method, bandwidth etc.
- **Many aspects only make sense to be fused in the World Coordinate Frame.**
- **E.g. absolute velocity of measured objects – if a leading vehicle does not move in the radar frame, how fast is that vehicle**
  - if the ego vehicle is standing?
  - if the ego vehicle is going 200kph?
- **We will concentrate on coordinate frames for now, and leave temporal coordination as an exercise.**





- Vector $\mathrm{v} \in R^n$:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

- Matrix $\mathbf{A} \in R^{m \times n}$:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

- Multiplying a vector $\mathbf{v} \in R^n$ with a matrix $\mathbf{A} \in R^{m \times n}$:

$$\mathbf{w} = \mathbf{A}\mathbf{v} = \begin{pmatrix} a_{11}v_1 + a_{12}v_2 + \dots + a_{1n}v_n \\ a_{21}v_1 + a_{22}v_2 + \dots + a_{2n}v_n \\ \vdots \\ a_{m1}v_1 + a_{m2}v_2 + \dots + a_{mn}v_n \end{pmatrix}$$

Only defined if dimensions match, i.e. the number of columns in the matrix is equal to the number of rows in the vector!

- Multiplying two matrices $\mathbf{A} \in R^{m \times n}$ and $\mathbf{B} \in R^{n \times m}$:

$$\mathbf{C} = \mathbf{AB} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} + \ldots + a_{1n}b_{n1} & a_{11}b_{12} + a_{12}b_{22} + \ldots + a_{1n}b_{n2} & \ldots & a_{11}b_{1m} + a_{12}b_{2m} + \ldots + a_{1n}b_{nm} \\ a_{21}b_{11} + a_{22}b_{21} + \ldots + a_{2n}b_{n1} & a_{21}b_{12} + a_{22}b_{22} + \ldots + a_{2n}b_{n2} & \ldots & a_{21}b_{1m} + a_{22}b_{2m} + \ldots + a_{2n}b_{nm} \\ \ldots & \ldots & \ddots & \ldots \\ a_{m1}b_{11} + a_{m2}b_{21} + \ldots + a_{mn}b_{n1} & a_{m1}b_{12} + a_{m2}b_{22} + \ldots + a_{mn}b_{n2} & \ldots & a_{m1}b_{1m} + a_{m2}b_{2m} + \ldots + a_{mn}b_{nm} \end{pmatrix}$$

  Only defined if dimensions match, i.e.
  - the number of columns in matrix **A** is equal to the number of rows in matrix **B**
  - and the number of rows in matrix **A** is equal to the number of columns in matrix **B**!
- Matrix multiplication is **associative**: **A(BC)=(AB)C**, but **not commutative: AB != BA**
- Special matrix: Unity matrix (or submatrix)

$$\mathbf{I^{n \times n}} = \begin{pmatrix} 1 & 0 & \ldots & 0 \\ 0 & 1 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 1 \end{pmatrix}$$

2D Rotation

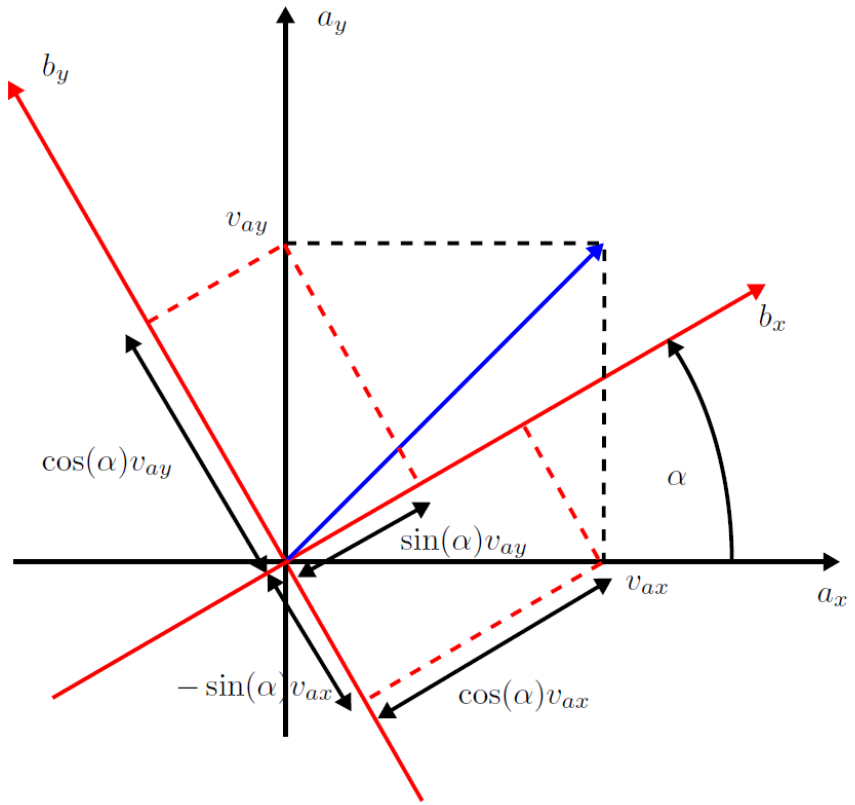
- We want to express a vector $v_a = \begin{bmatrix} v_{ax} & v_{ay} \end{bmatrix}^T$ given in frame a in the coordinates of the frame b: $v_b = \begin{bmatrix} v_{bx} & v_{by} \end{bmatrix}^T$
- Frame b is rotated with respect to frame a by an angle α
- Compute the projection of v_{ax} and v_{ay} on the axis of frame b

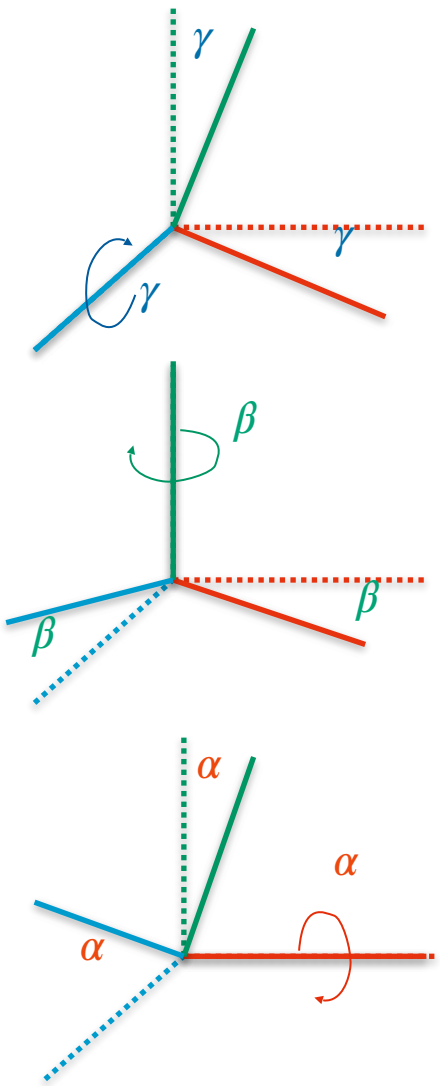
$$v_{b,x} = \cos(\alpha)v_{a,x} + \sin(\alpha)v_{a,y}$$
$$v_{b,y} = -\sin(\alpha)v_{a,x} + \cos(\alpha)v_{a,y}$$

- Transferring the above to matrix calculus we obtain

$$\boldsymbol{v}_b = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{bmatrix} \cdot \begin{bmatrix} v_{a,x} \\ v_{a,y} \end{bmatrix}$$

Rotational matrix $\boldsymbol{T}_a^b$ in 2D





- Elementary rotation by γ around z:

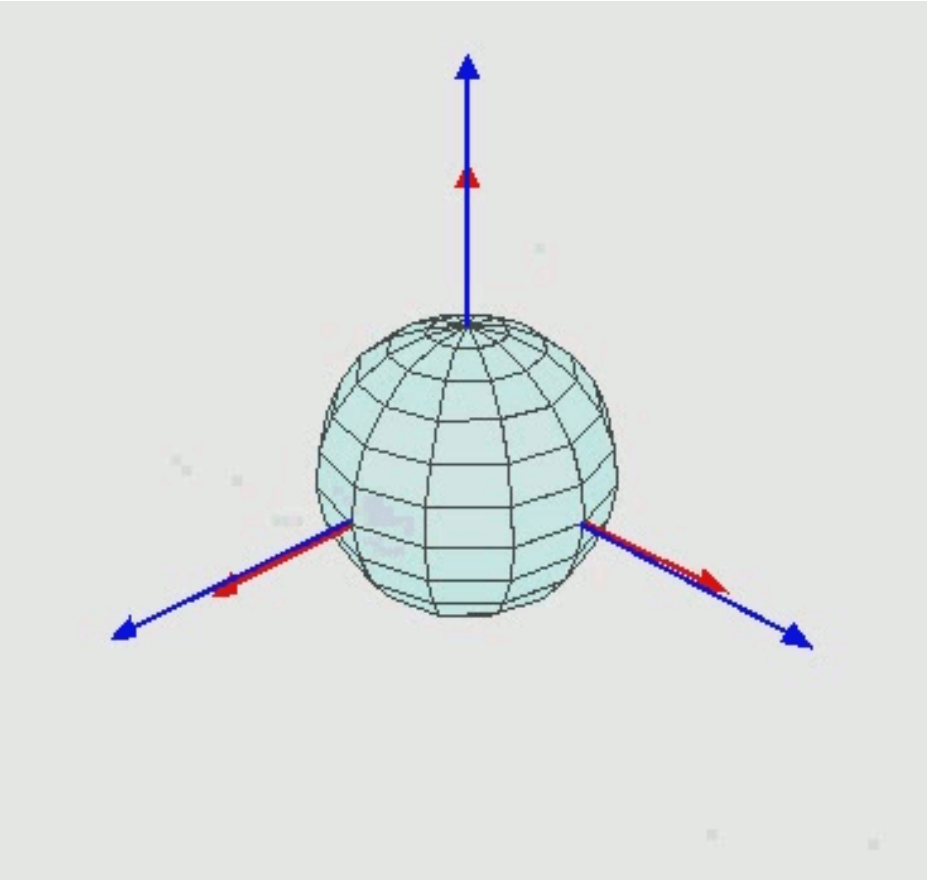
$$x' = x\cos\gamma - y\sin\gamma$$
$$y' = x\sin\gamma + y\cos\gamma$$
$$z' = z$$
$$\mathbf{x}' = \mathbf{R}_z\mathrm{x} = \begin{pmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}\mathbf{x}$$

- Elementary rotation by β around y:

$$x' = x\cos\beta + z\sin\beta$$
$$y' = y$$
$$z' = -x\sin\beta + z\cos\beta$$
$$\mathbf{x}' = \mathbf{R}_y\mathrm{x} = \begin{pmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{pmatrix}\mathrm{x}$$

- Elementary rotation by α around x:

$$x' = x$$
$$y' = y\cos\alpha - z\sin\alpha$$
$$z' = y\sin\alpha + z\cos\alpha$$
$$\mathbf{x}' = \mathbf{R}_y\mathbf{x} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{pmatrix}\mathrm{x}$$



Classical Euler angles -- Source: Wikipedia

- **Any rotation in 3D space can be expressed as the product of 3 successive rotations.**
- **This concept is known as „Euler Angles“ after Leonhard Euler (who didn‘t discover this fact but described it scientifically first in 1775.**
- **There is no single norm on how to do this. Rather, there are several established variants.**
- **Classical Euler angles:**

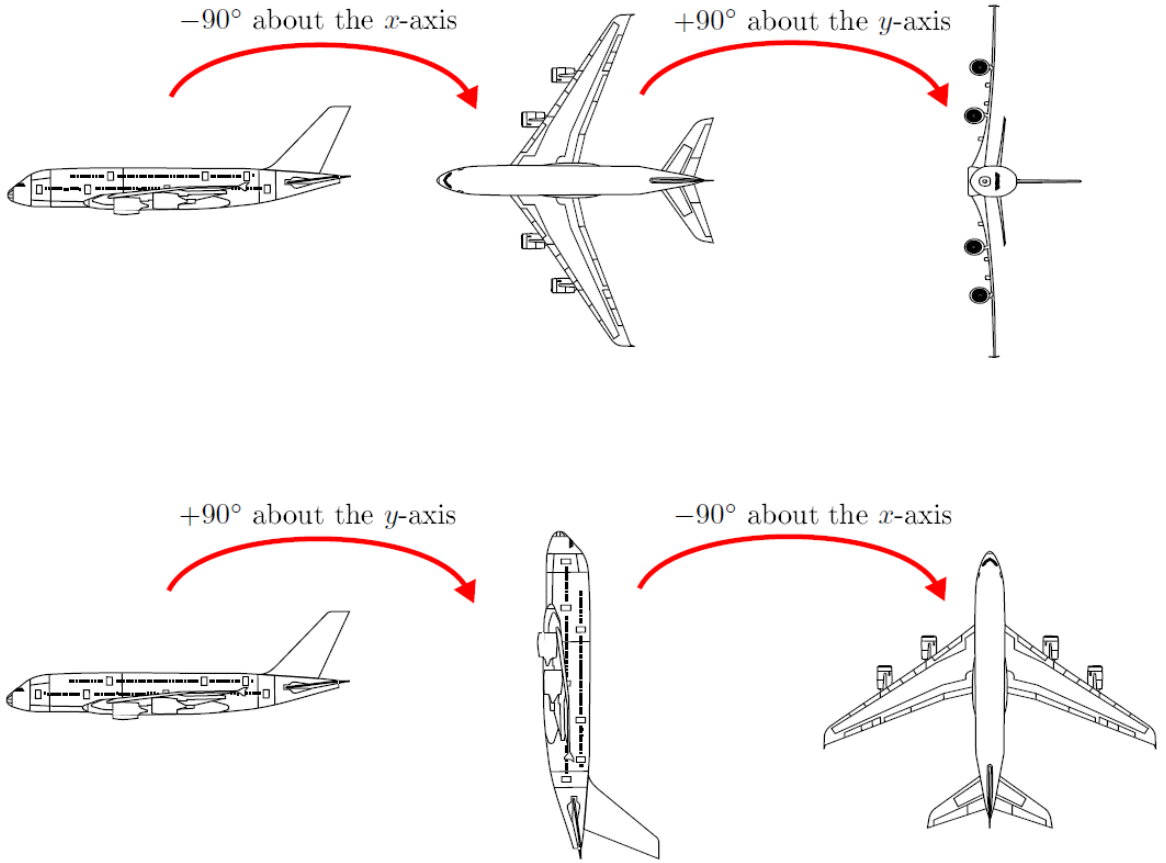
  1. Rotate around z axis by γ_1
  2. Rotate around (the new) x‘ axis by α
  3. Rotate around (the new) z“ axis by γ_2

  $$\mathbf{R} = \mathbf{R}_{z''}\mathbf{R}_{x'}\mathbf{R}_{z}$$

- Important to note that

$$\boldsymbol{T}_f = \boldsymbol{T}_z \cdot \boldsymbol{T}_y \cdot \boldsymbol{T}_x \neq \boldsymbol{T}_x \cdot \boldsymbol{T}_y \cdot \boldsymbol{T}_z$$

- This can be easily verified graphically
- Apply rotations of $\pm 90°$, start from the same attitude
- Sequence 1:
  - First rotate by $-90°$ about the x-axis
  - Then rotate by $+90°$ about the y-axis
- Sequence 2:
  - First rotate by$+90°$ about the y-axis
  - Then rotate by $-90°$ about the x-axis



Gimbal-Lock Phenomenon of Euler Angles

- Euler angles have a singularity at $\theta = \pm \pi/2$
- Considering the sequence of rotations from the n.e.d. to the body-fixed frame for $\theta = \pi/2$ we have

$$
\begin{aligned}
\boldsymbol{T}_n^b &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\Phi & \sin\Phi \\ 0 & -\sin\Phi & \cos\Phi \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \cos\Psi & \sin\Psi & 0 \\ -\sin\Psi & \cos\Psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 0 & 0 & -1 \\ \sin\Phi\cos\Psi - \cos\Phi\sin\Psi & \cos\Phi\cos\Psi + \sin\Phi\sin\Psi & 0 \\ \cos\Phi\cos\Psi + \sin\Phi\sin\Psi & \cos\Phi\sin\Psi - \sin\Phi\cos\Psi & 0 \end{bmatrix} \\
&= \begin{bmatrix} 0 & 0 & -1 \\ \sin(\Phi - \Psi) & \cos(\Phi - \Psi) & 0 \\ \cos(\Phi - \Psi) & -\sin(\Phi - \Psi) & 0 \end{bmatrix}
\end{aligned}
$$

- There is an infinite number of combinations of Φ and Ψ representing the same transformation matrix
- No unique mapping from the transformation matrix to the Φ and Ψ

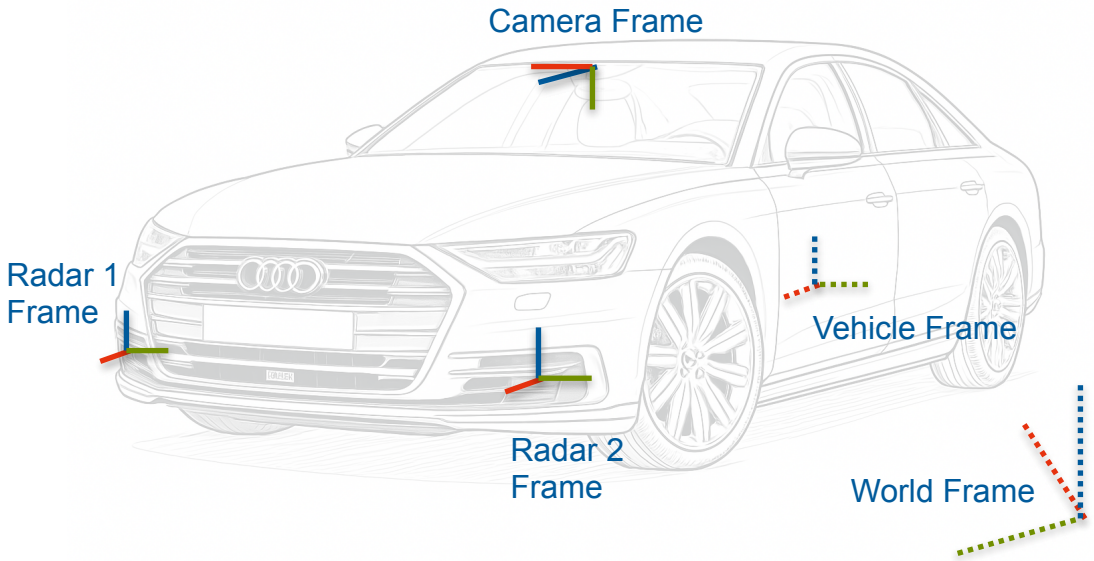
- **Putting translation and rotation together, the full coordinate transform becomes**
  $\mathbf{x}' = \mathbf{R}\mathbf{x} + \mathbf{t}$
  with the Euler rotation matrix R and the translation vector t describing the relationship between coordinate frames.

- **This expression can be simplified by extending coordinates to 4 dimensions by adding a 1:**

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- **Then we can combine R and t like this:**

$$\mathbf{M} = \begin{pmatrix} & \mathbf{R} & & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



- **...and the full equation becomes**

$$\mathbf{x}' = \mathbf{M}\mathbf{x}$$

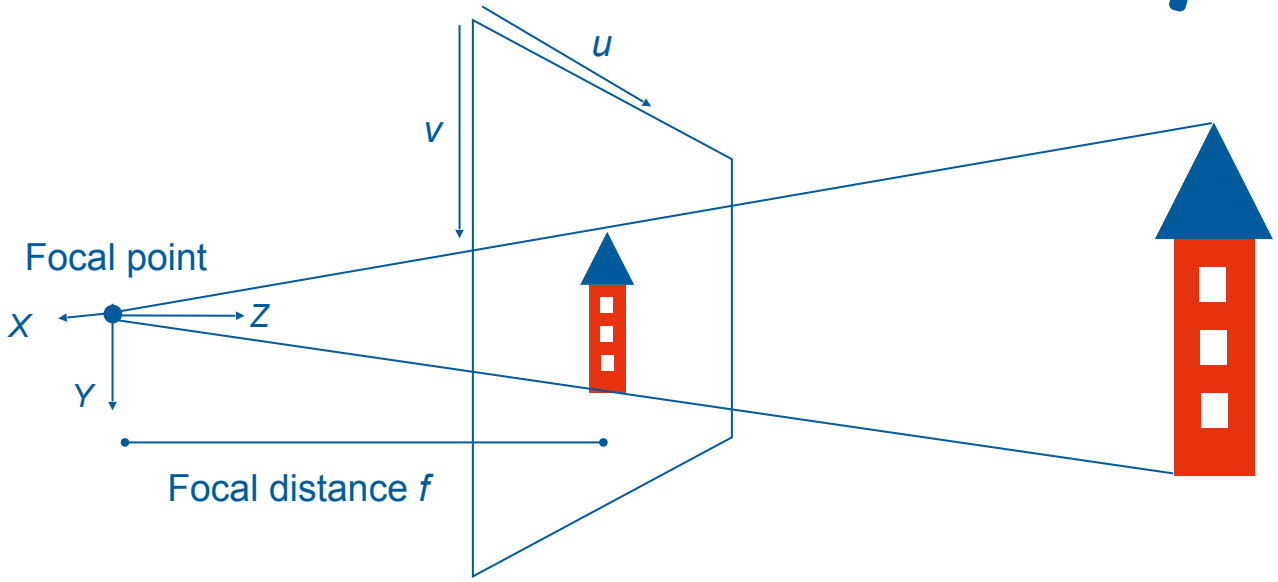
- **We can now easily chain multiple transforms (e.g. world -> vehicle with M_v, vehicle -> camera with M_c):**

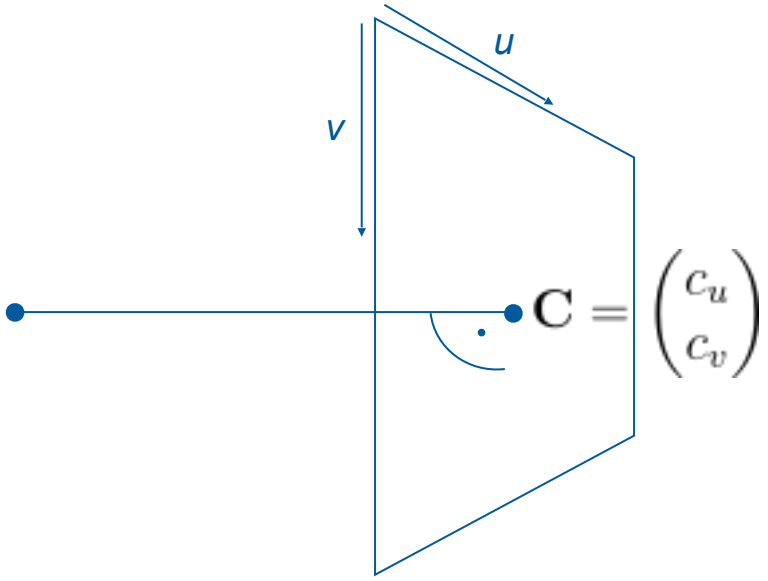
$$\mathbf{x}' = \mathbf{M}_v\mathbf{M}_c\mathbf{x}$$

Cameras, Projection, Calibration

- **Cameras are inherently 2D sensors. The 3D environment is projected onto the imager, losing distance information.**
- **If we want to model the world with cameras,**
  - we must understand how camera projection works, and
  - we must be able to calibrate our camera for its specific calibration parameters.
- **The simplest camera model is the pinhole model, with three parameters:**
  - Focal distance *f*
  - Image center $\mathbf{C} = \begin{pmatrix} c_u \\ c_v \end{pmatrix}$

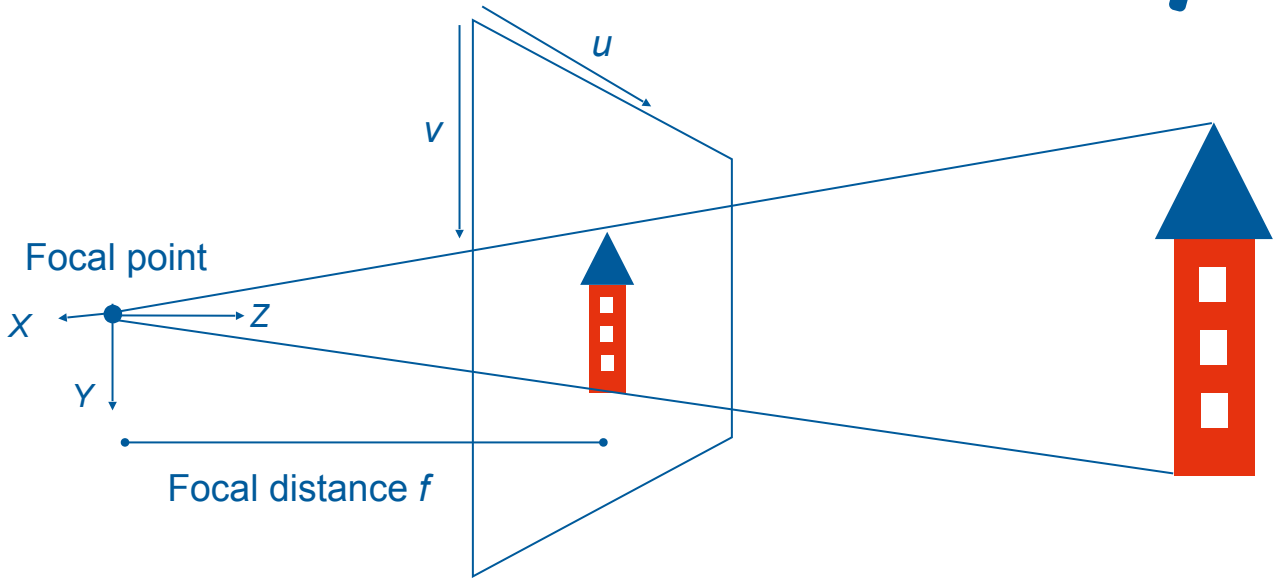
    (the point which the ray with x=0,y=0 projects to)





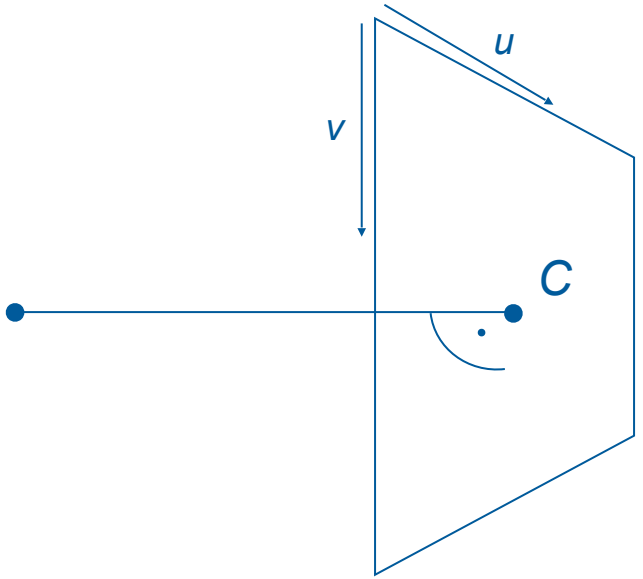
- **The basic projections in the pinhole model follow the intercept theorem („Strahlensatz"):**

$$\frac{x}{z} = \frac{u + c_u}{f} \Rightarrow u = f\frac{x}{z} - c_u$$
$$\frac{y}{z} = \frac{v + c_v}{f} \Rightarrow v = f\frac{y}{z} - c_v$$



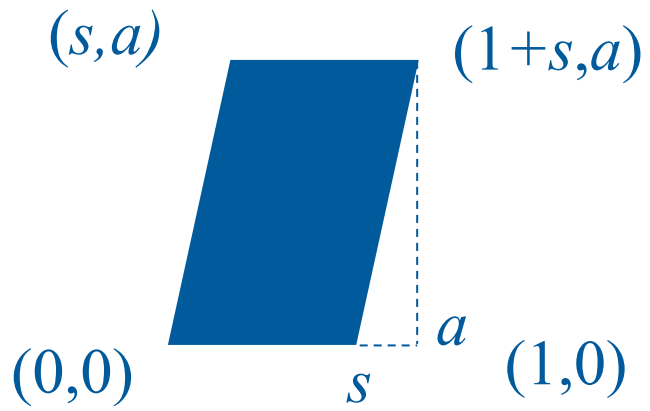
- **...or as a vector-matrix equation:**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} f & 0 & -c_u \\ 0 & f & -c_v \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$



- **Often camera pixels are not exactly square, especially with cheap cameras, webcams or similar..**
- **Some imagers actually have rectangular pixels.**
- **Often the imager chip is not precisely aligned with the lens.**
- **It is helpful to extend the calibration by a factor a that relates this ratio:**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} f & 0 & -c_u \\ 0 & af & -c_v \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$



- **The other element of camera projection is the camera‘s coordinate frame, given as**

$$\mathbf{M} = \begin{pmatrix} & \mathbf{R} & & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where R is the 3x3 rotation matrix and t is the 3-vector modeling the translation.

- **The complete projection equation becomes**

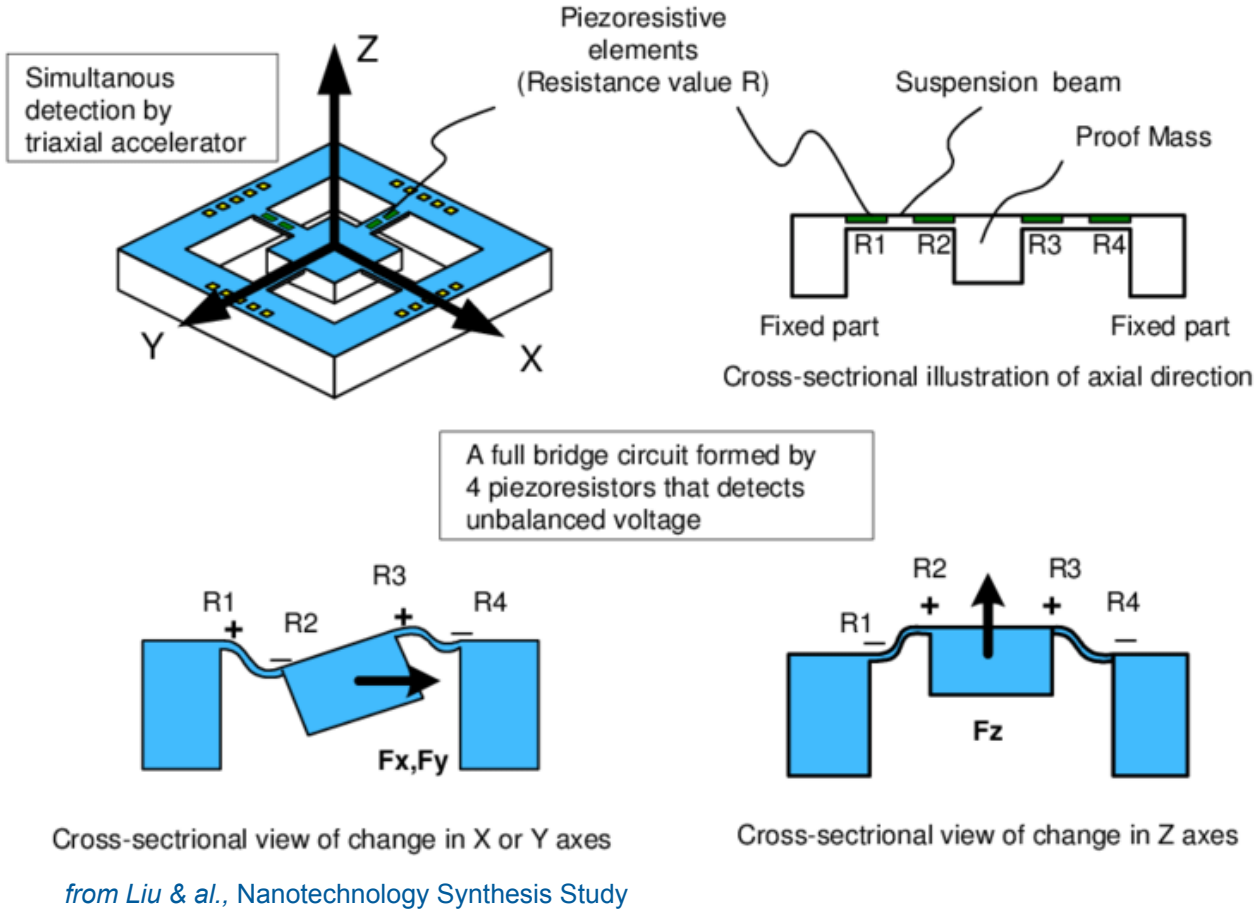
$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = (\mathbf{KR} \quad \mathbf{Kt}) \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- **We therefore need to calibrate 9 parameters, or 12 if we ignore the constraints contained in the rotation matrix R.**

Simple Fusion Example: Pitch and Roll from Gyro and Accelerometer

(with Complementary Filters, for now)

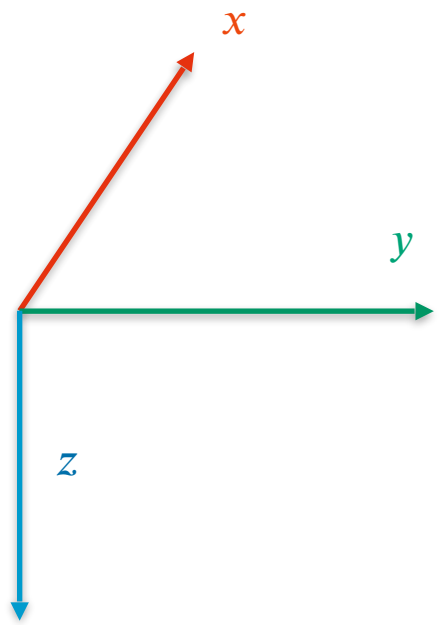
- Task: **Estimate Roll and Pitch for a Robot** (or vehicle, or drone, or or or...)
- Inputs:
  - Accelerometer – Measures **translational acceleration** (e.g. gravity, but also all others)
  - Gyroscope – Measures rotational **velocity** (but no absolute rotation)
- Desired output:
  - 2 degrees of freedom: pitch and roll
  - Free of drift, reasonably free of noise, absolute
- Limit: Can not be absolute in heading without a magnetometer (see later)



from Liu & al., Nanotechnology Synthesis Study

Accelerometer Output

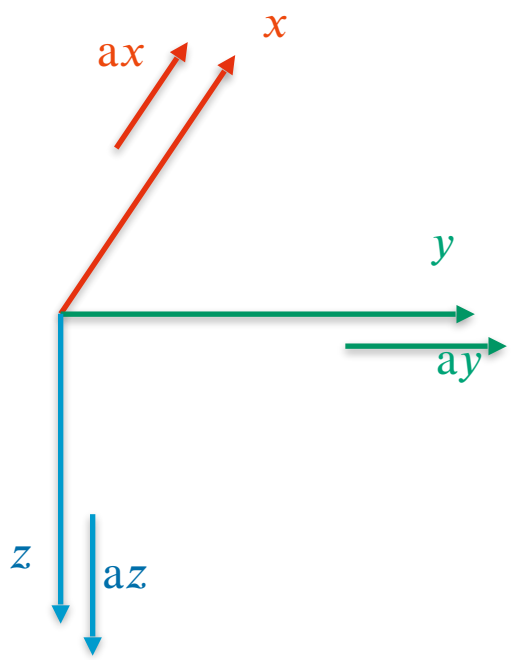
- **Glossary:**
  - Position of a body in space $\mathbf{x} = (x, y, z)$
  - Velocity of a body through space $\mathbf{v} = (vx, vy, vz) = \frac{\partial \mathrm{x}}{\partial t}$
  - Acceleration of a body through space $\mathbf{a} = (ax, ay, az) = \frac{\partial \mathrm{v}}{\partial t} = \frac{\partial 2\mathrm{x}}{\partial t2}$
  - Gravity vector $\mathbf{g} = (0{,}0, g)$
  - Estimate value: marked by a „hat“ $\hat{a}$
  - Derivative: marked by a dot $\dot{\alpha}$
- **In this example, we are using a coordinate system where z is pointing down to reflect the direction of gravity. As we know, adding a 180° rotation around x at the end will right us up again, so we save it here.**



- **Let‘s start with the accelerometer. How can we describe what it measures?**

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \frac{d\mathbf{v}}{dt} + \mathbf{w}_b \times \mathbf{v} - \mathbf{R}\begin{pmatrix} 0 \\ 0 \\ g \end{pmatrix} + \mathbf{b}(t) + \mathbf{n}(t)$$

Measured Acceleration	Linear motion	Rotational motion	Gravity	Time-variant bias	Time-variant noise



- **How do we get roll/pitch estimates, i.e. the matrix R from this?**
- **Let‘s simplify. If the accelerometer is at rest, and we ignore bias and noise, we see**

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = R_{\alpha,x} R_{\beta,y} \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} = \begin{bmatrix} g\sin\beta \\ -g\sin\alpha\cos\beta \\ g\cos\alpha\cos\beta \end{bmatrix}$$

- **With** $\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \begin{pmatrix} g \sin\beta \\ -g \sin\alpha \cos\beta \\ g \cos\alpha \cos\beta \end{pmatrix}$ **, we can simplify to**

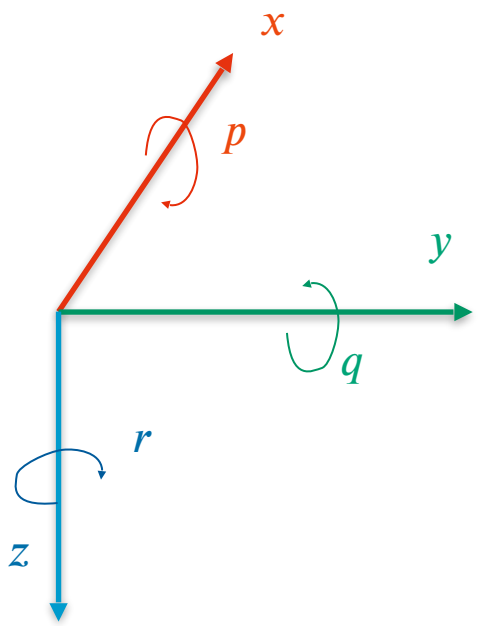
$$\frac{a_y}{a_z} = -\frac{\sin\alpha}{\cos\alpha} = -\tan\alpha \qquad \hat{\alpha}_{\text{acc}} = \tan^{-1}\frac{a_y}{a_z}$$

$$\frac{a_x}{g} = \sin\beta \qquad \hat{\beta}_{acc} = \sin^{-1}\frac{a_x}{g}$$

- **However, this ignores all the other acceleration terms:**
  - actual linear and angular acceleration (kind of important for a vehicle...)
  - time-variant bias due to changes in temperature, humidity etc. – can we calibrate that out?
  - noise – this is typically high frequency and can be reduced by high pass filtering.

- **Let‘s look at the gyroscope now. It outputs rates of rotation around the body axes.**

$$\begin{pmatrix} \hat{p} \\ \hat{q} \\ \hat{r} \end{pmatrix} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} + \mathbf{b}(t) + \mathbf{n}(t)$$



- **We need to transform to Euler rates to get this into the inertial (world) frame:**

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \end{pmatrix} = \begin{pmatrix} 1 & \sin\alpha\tan\beta & \cos\alpha\tan\beta \\ 0 & \cos\alpha & -\sin\alpha \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix}$$

(Why? → exercise! ☺)

...for which we already need to know α and β...

- **Let‘s simplify and look at the case of initial** $\alpha = 0, \beta = 0$
  The equation becomes

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \end{pmatrix} = \begin{pmatrix} 1 & \sin 0 \tan 0 & \cos 0 \tan 0 \\ 0 & \cos 0 & -\sin 0 \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} p \\ q \end{pmatrix}$$

- **We could simply integrate from here:**

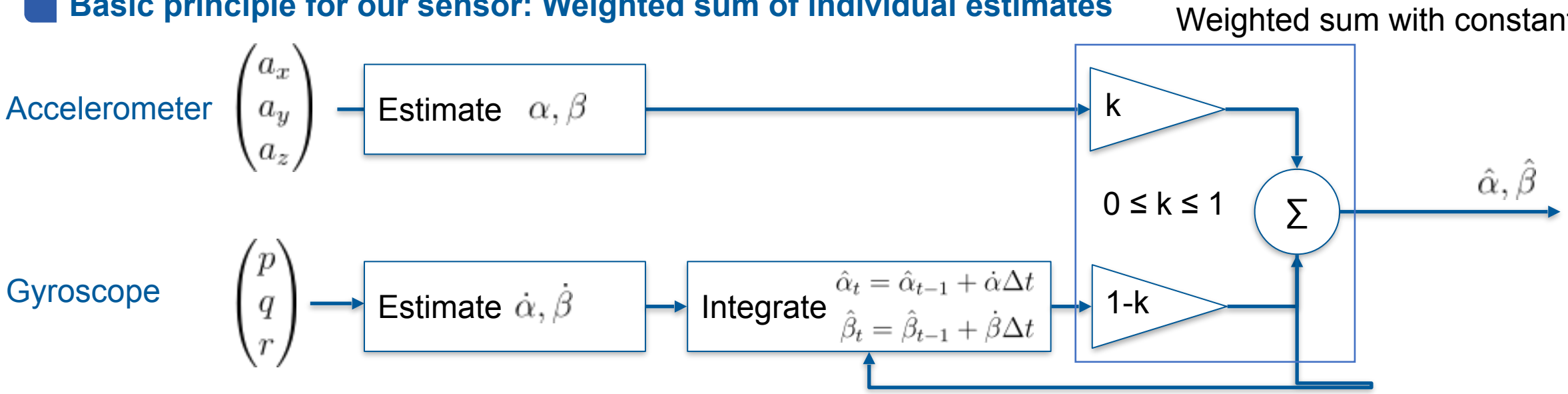
$$\alpha = \int_0^T \dot{\alpha} dt \qquad \beta = \int_0^T \dot{\beta} dt$$

or discretely

$$\hat{\alpha}_t = \hat{\alpha}_{t-1} + \dot{\alpha}\Delta t$$
$$\hat{\beta}_t = \hat{\beta}_{t-1} + \dot{\beta}\Delta t$$

- **Both accelerometer and gyroscope can give estimates of pitch and roll at certain points.**
  - Accelerometers give good estimates when at rest or over very long periods of time, when accelerations average out and only gravity is left.
  - Gyroscopes give good estimates over very short periods of time, when noise and bias have no time to accumulate.
- **We cannot solve the problem with either of the sensors alone. We need to fuse the data.**

- **Complementary filters are one of the simplest and most computationally effective fusion methods.**
- **The idea is to use the output of one sensor to correct errors in the output of the other, and vice versa.**
- **Basic principle for our sensor: Weighted sum of individual estimates**



- **Complementary Filter equation:**

$$\hat{\alpha}_n = k\hat{\alpha}_{\text{acc},n} + (1-k)(\hat{\alpha}_{n-1} + \dot{\alpha}_{\text{gyro},n}\Delta t)$$
$$\hat{\beta}n = k\hat{\beta}_{\text{acc},n} + (1-k)(\hat{\beta}_{n-1} + \dot{\beta}_{\text{gyro},n}\Delta t)$$

- **An alternative way of thinking about the Complementary Filter:**
- **We can think of the development of the state of a system that follows a linear, time-invariant model like this:**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1}$$

where

- x is the state vector,
- A is the system model, i.e. “the way things change in the system over time if there is no input“,
- u is a vector of inputs to the system,
- and B is the way how these inputs are influencing our system state.

- **If there were no noise and we could just integrate the gyro, our model can be written as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t & 0\\ 0 & \Delta t\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}}$$

- **However, we know that that is not correct, so we introduce a correction term...**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1} + \mathbf{L}(\mathbf{x}_{n-1} - \mathbf{x}_{\mathrm{meas},\mathbf{n-1}})$$

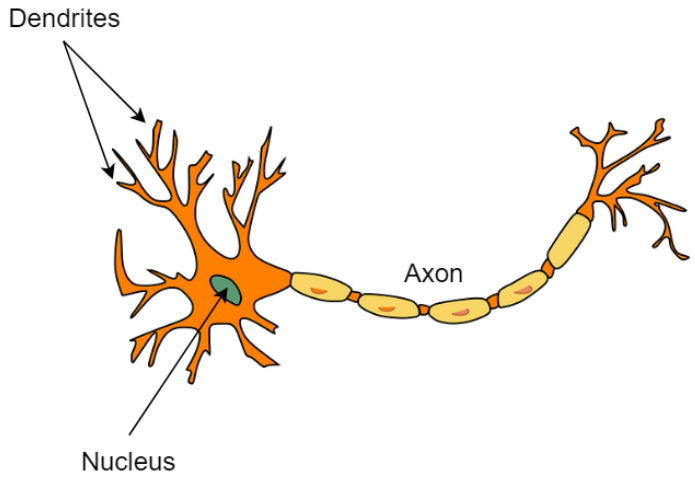
...again with a model L that governs how the correction term applies to the system state.

- **We can write this now as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t(1-k) & 0\\ 0 & \Delta t(1-k)\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}} + \underbrace{\begin{pmatrix}k & 0\\ 0 & k\end{pmatrix}}_{\mathbf{L}} \begin{pmatrix}\hat{\alpha}_n - \hat{\alpha}_{\mathrm{acc},n-1}\\ \hat{\beta}_n - \hat{\beta}_{\mathrm{acc},n-1}\end{pmatrix}$$

- **The above form is known as a *Luenberger Observer* and is a classic in control theory.**
- **Compare to the Kalman Filter!**

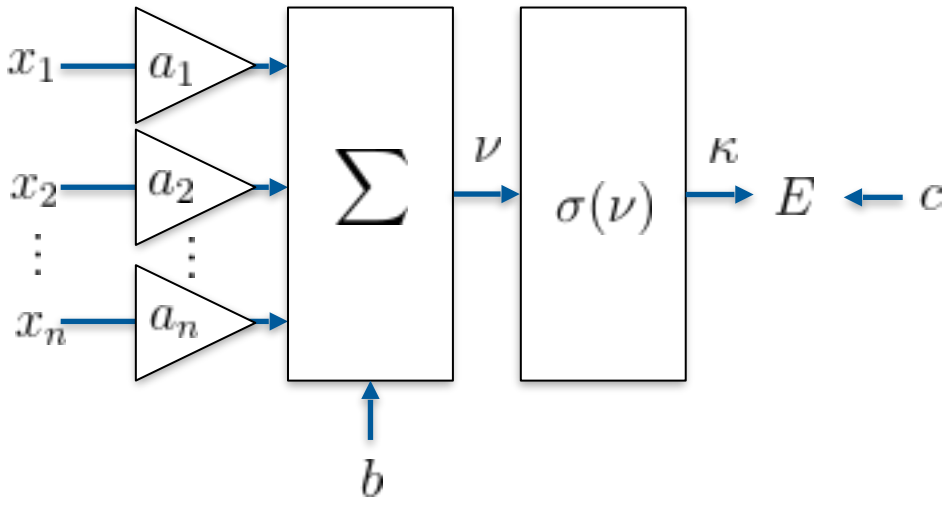
Machine Learning: Neurons Gradient Descent



- **There are many classification algorithms: decision trees, k-nearest neighbor clustering, support vector machines, ...**
- **What is commonly called AI is built on the Artificial Neuron.**
- **It is modelled after a biological neuron, which**
  - collects electrical impulses in its dendrites
  - produces an output in its nucleus and forwards it through its axon,
  - which connects to other neurons‘ dendrites.
- **The AN produces the output**

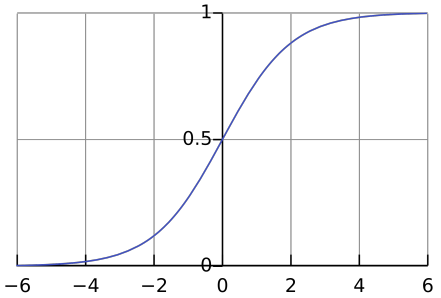
$$\kappa = \sigma(\sum_i x_i a_i + b) = \sigma(\mathbf{x}\mathbf{a}^\top + b)$$

- **In fixed networks, the weights and biases a_i, b form the *model.***
- **σ is the activation function, with several classical options.**



- **One of the most common activation functions is the *sigmoid function***

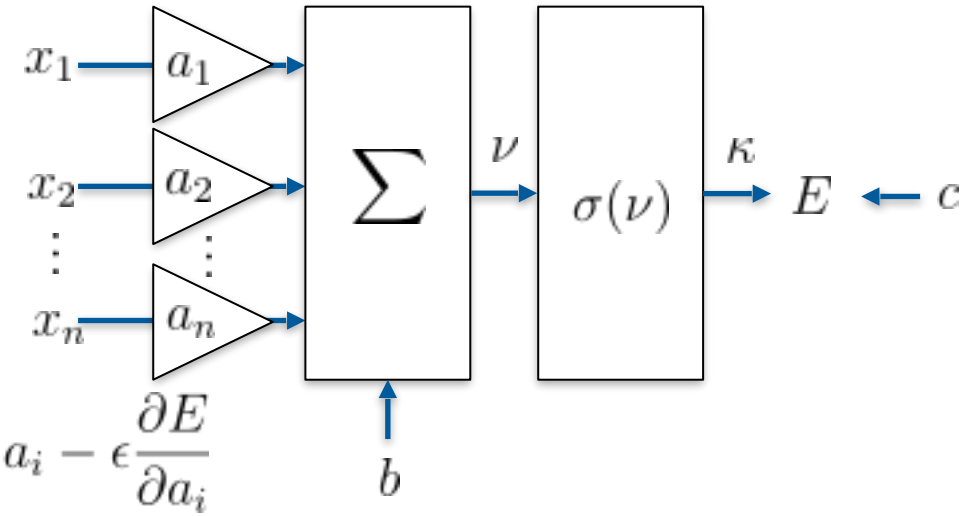
$$\sigma(\nu) = \frac{1}{1 + e^{-\nu}}$$



  It is continuous and differentiable, approaches a binarization function for sufficiently large input values, and its output can be easily interpreted by developers.

- **If we want to train a single neuron, we need to adapt its weights according to the error of its output. We can calculate the absolute error as**

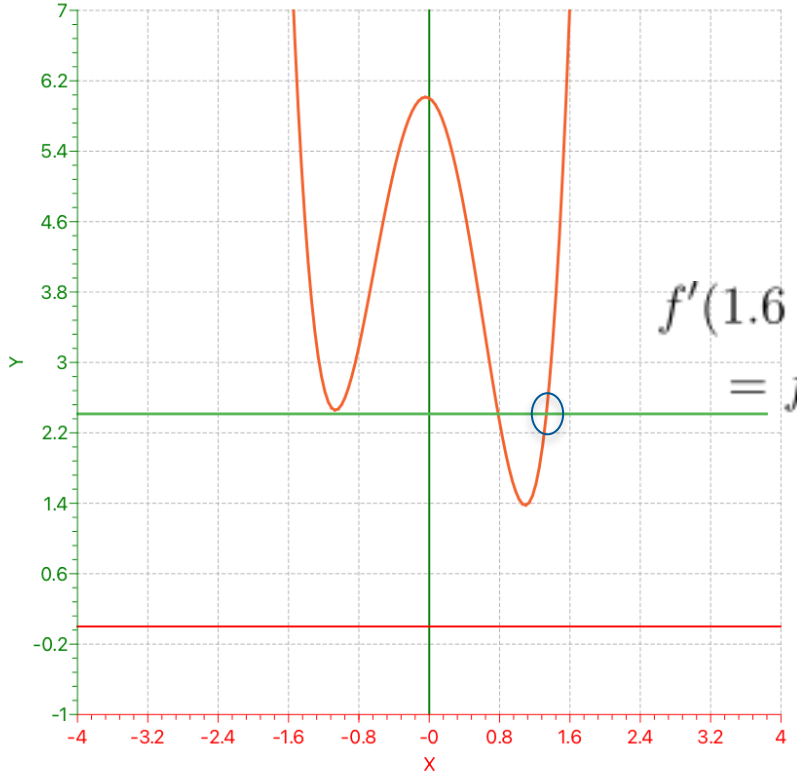
$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$



- **To break this down to the individual weights, we calculate the partial derivatives $\frac{\partial E}{\partial a_i}$ and apply them to the weights incrementally with a scaling factor ϵ: $a_i' = a_i - \epsilon \frac{\partial E}{\partial a_i}$**

- **The scaling factor is usually very small to *descend along the gradient* in the right direction.**

Sensor Networks and Sensor Fusion
Gradient Descent

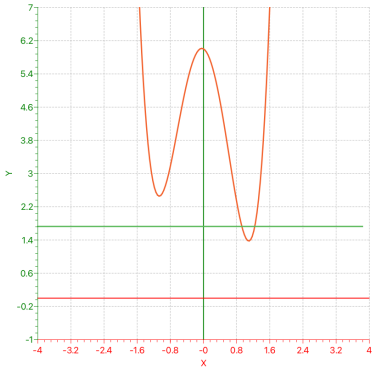


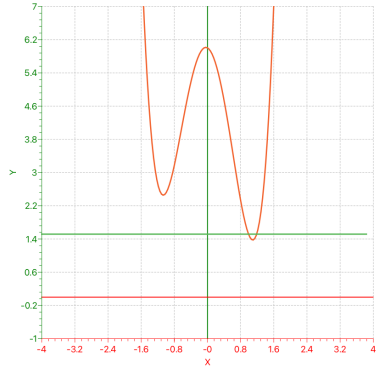
$$f'(1.6 - 26.25\epsilon) = f'(1.3375) \approx 9.5$$

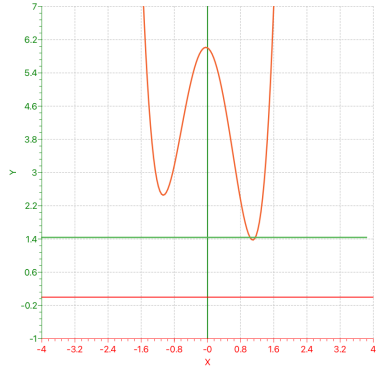
$$f'(1.3375 - 9.5\epsilon) = f'(1.2425) \approx 5.12$$

$$f'(1.1425 - 5.12\epsilon) = f'(1.1913) \approx 3.11$$

$$f'(1.1425 - 3.11\epsilon) = f'(1.1602) \approx 2$$







$$f'(1.14) \approx 1.32$$
$$f'(1.127) \approx 0.99$$
$$f'(1.117) \approx 0.58$$
$$f'(1.111) \approx 0.4$$
$$\vdots$$
$$f'(1.09756) \approx 0$$
$$f(1.09756) \approx 1.3722$$

We arrive at a local minimum of 1.3722 at position x=1.09756.

For the multi-variable case $f(x_1, x_2, \ldots, x_n)$**,**

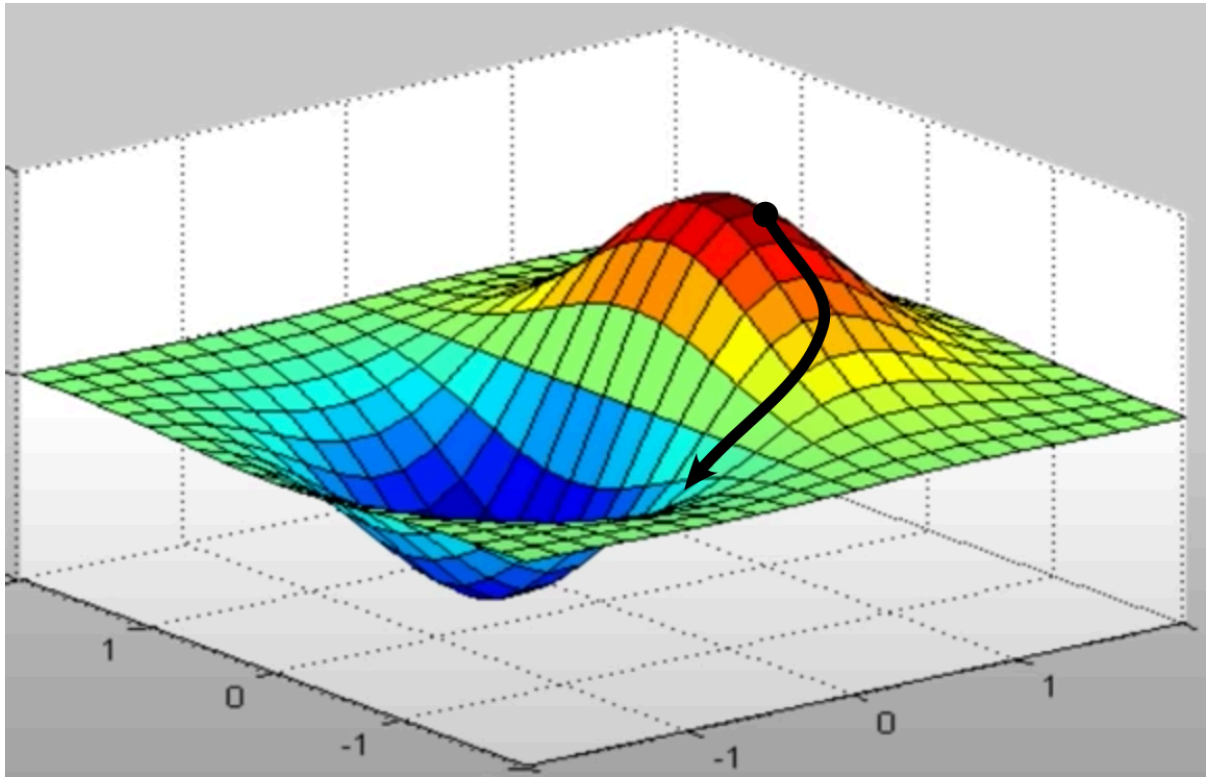
we need the *gradient vector,* i.e. the vector of partial derivatives

$$f'(x_1, x_2, \ldots, x_n) = \nabla f = \begin{pmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \vdots \\ \partial f/\partial x_n \end{pmatrix}$$

We start at a random point in *n*-dimensional space and calculate ∇f **.**

We need to *descend in the direction of* $-\nabla f$**.**

The *length of* $-\nabla f$ **gives us an indication of how much we need to descend.**

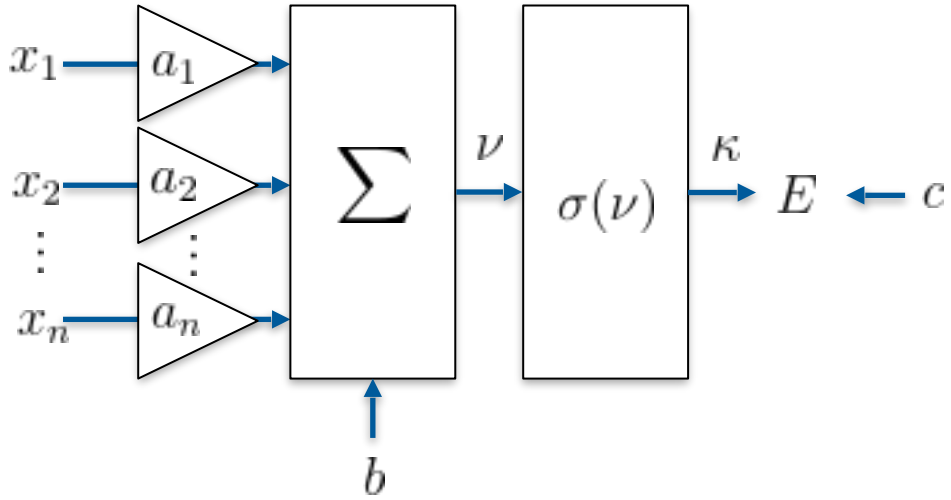


- **Let‘s look at a single neuron.**
- **We want to find the *minimum of the error function***

$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$

using gradient descent. So we need the gradient vector

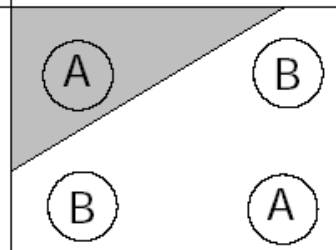
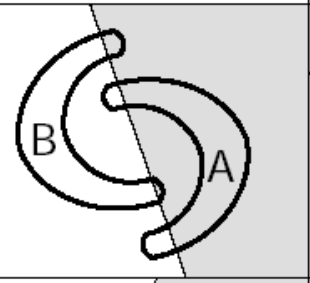
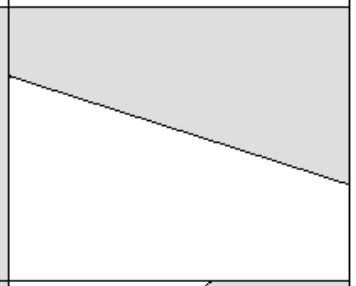
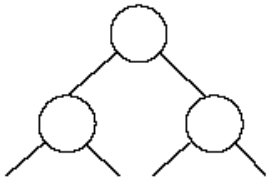
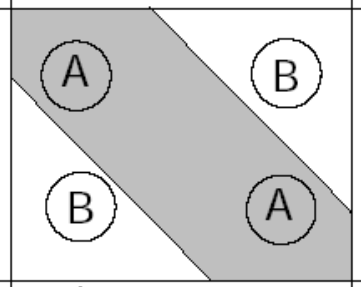
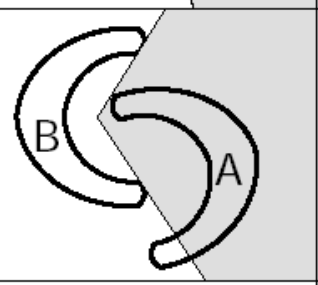
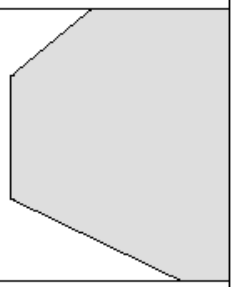
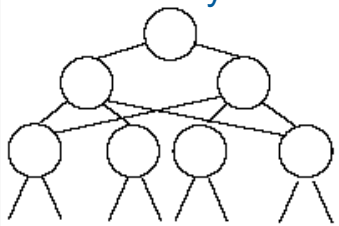
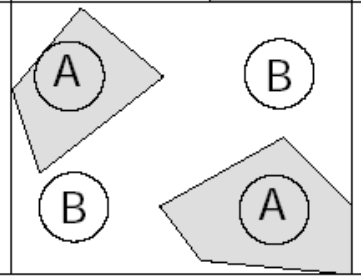
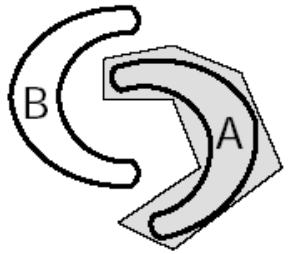
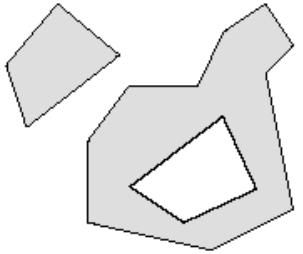
$$\nabla E = \begin{pmatrix} \partial E/\partial a_1 \\ \partial E/\partial a_2 \\ \vdots \\ \partial E/\partial a_n \end{pmatrix}$$



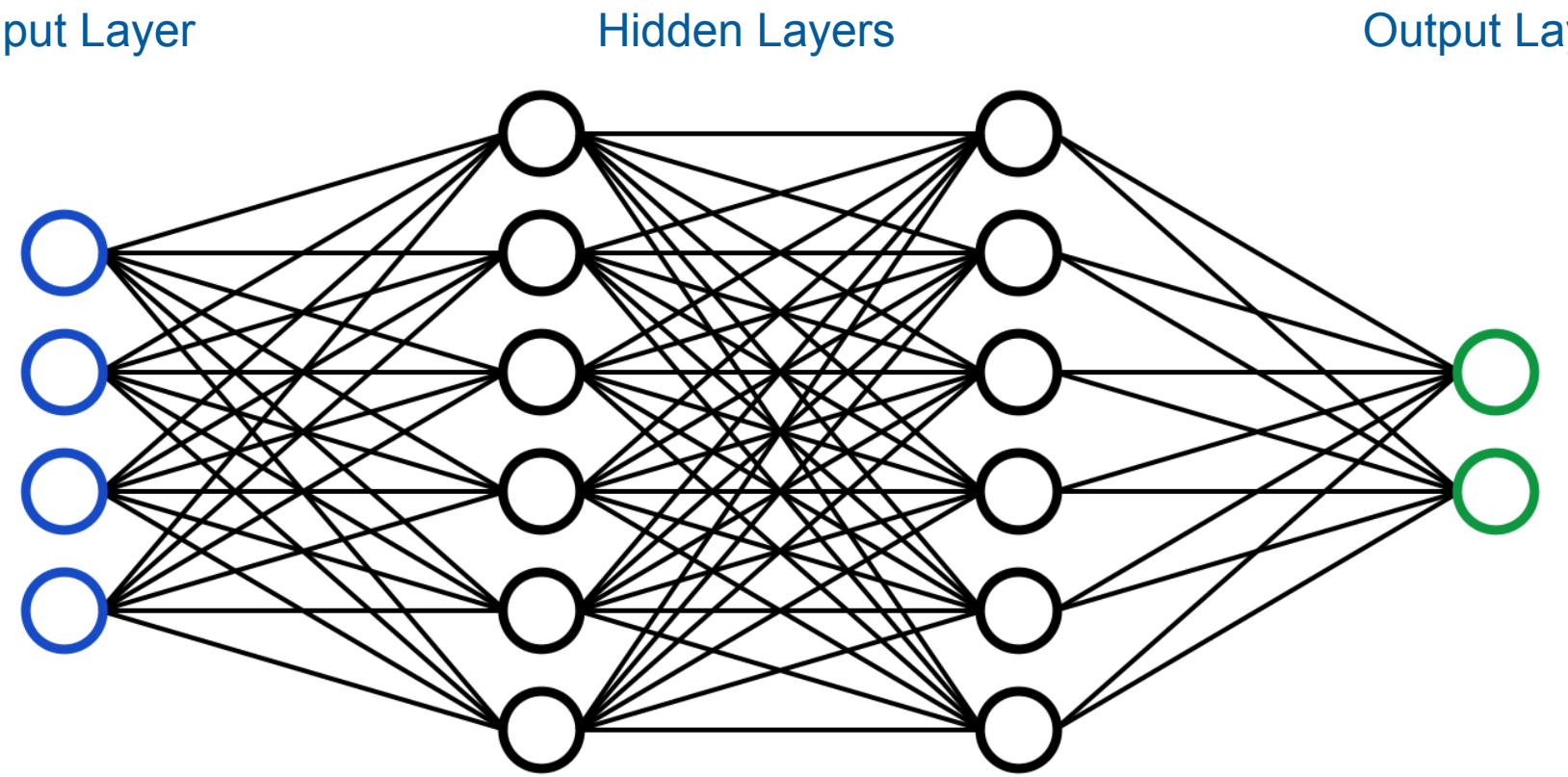
- **Applying the chain rule twice, the individual components of the gradient vector are**

$$\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$$

- **A single neuron models a single line (or hyperplane) separator. It can for example not model XOR.**
- **Like in a human brain, we construct multiple layers of neurons for more general separators.**

Network Structure	Separation Region	XOR separation	Nested Region Separation	General Region Separation
One layer 	Single plane			
Two layers 	Convex regions			
Three layers 	Arbitrarily complex regions			

For more difficult problems, we build up multi-layer networks of many neurons.



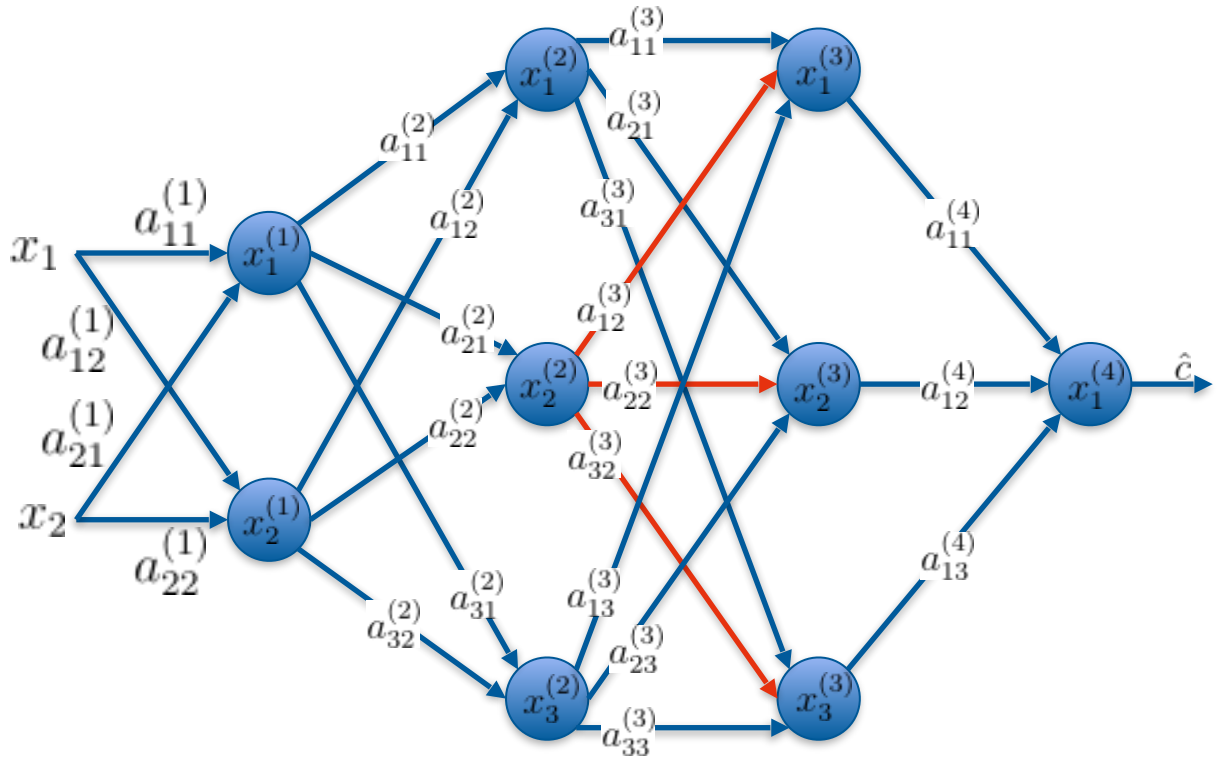
- **Let‘s look at a network with a bit more complex topology.**
- **The derivation for the nodes in the output layer remains the same –**

  the weights $a_{jk}^{(L)}$ are only influenced by $\hat{c} = x_i^{(L)}$.
- **To backpropagate, we need to consider that the local error is caused by influences coming down multiple paths:**

$$\frac{\partial E}{\partial a_{jk}^{(L)}} = \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial a_{jk}^{(L)}}$$

becomes

$$\frac{\partial E}{\partial x_k^{(L-1)}} = \sum_{j=0}^{n_{L-1}} \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial x_k^{(L-1)}}$$



- **For each example m in the training set, we can calculate an error E_m by forward-calculating the network output and subtracting it from the output pattern the network is supposed to approximate.**
- **We are interested in the performance of the network over the full training set, which we can average out:**

$$E_{\text{avg}} = \frac{1}{n}\sum_{m=1}^{n} E_m$$

$$\frac{\partial E_{\text{avg}}}{\partial x^{(L)}} = \frac{1}{n}\sum_{m=0}^{n} \frac{\partial E_m}{\partial x^{(L)}}$$

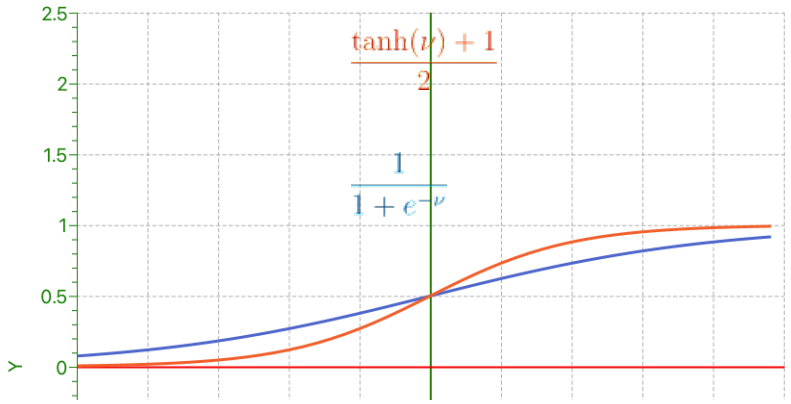
- **I.e. our gradient vector is computed as the average of the gradient vectors of the individual training examples.**

To conclude,

1. We initialize the network with random weights and biases, obtaining the vector
$$N = (a_{11}^{(1)}, a_{21}^{(1)}, \ldots, b_1, a_{12}^{(1)}, a_{22}^{(1)}, \ldots, b_2, \ldots, a_{jk}^{(N)}, b_k)^\top$$
This vector, together with the activation functions $\sigma_k^{(N)}$, describes our network.
2. We forward-compute the output error for each output neuron throughout the network by subtracting it from the setpoint output given by the training set.
3. We obtain the average error E_{avg} for all examples in the training set.
4. We compute the vector of partial derivatives as a function of the weights, biases, and average output error,
$$\nabla E = (\frac{\partial E}{\partial a_{11}^{(1)}}, \frac{\partial E}{\partial a_{21}^{(1)}}, \ldots, \frac{\partial E}{\partial b_1}, \frac{\partial E}{\partial a_{12}^{(1)}}, \frac{\partial E}{\partial a_{22}^{(1)}}, \ldots, \frac{\partial E}{\partial b_2}, \ldots, \frac{\partial E}{\partial a_{jk}^{(N)}}, \frac{\partial E}{\partial b_k})^\top$$
5. We descend the gradient given by ∇E „an ϵ-sized step“ by modifying the network:
$$N' = N - \epsilon \nabla E$$

Activation Functions

- **Sigmoid:** $\sigma(\nu) = \frac{1}{1+e^{-\nu}}$ $\quad \frac{\partial \sigma}{\partial \nu} = \sigma(\nu)(1-\sigma(\nu))$

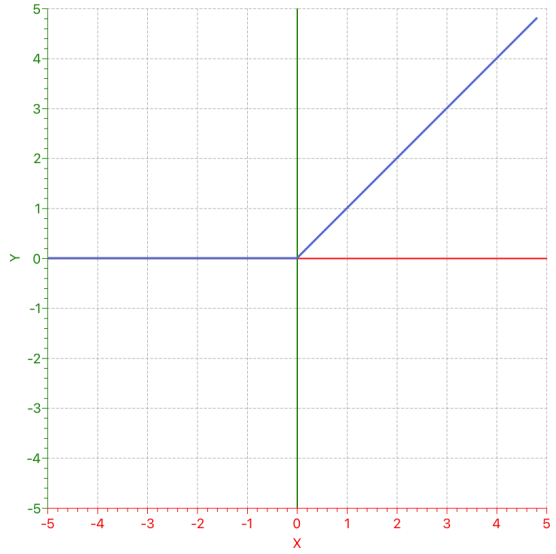


- **Tangens hyperbolicus:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{\partial}{\partial dx}\frac{e^x - e^{-x}}{e^x + e^{-x}} = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)^2 = 1 - \tanh(x)^2$$

- **ReLU:** $\mathrm{ReLU}(x) = max(0,x) = \frac{x+|x|}{2}; \frac{\partial \mathrm{ReLU}}{\partial x} = \begin{cases} 1 & : x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$



Measuring Quality

Precision and Recall: Classification Quality

- ***Precision*** measures *how many classifications are correct*:

$$\text{Precision} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifications}} = \frac{tp}{tp + fp}$$

- ***Recall*** measures *how many instances were identified:*

$$\text{Recall} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifiable Objects}} = \frac{tp}{tp + fn}$$

- Example:
  - An image contains 10 cars.
  - 7 were correctly identified.
  - 2 non-car patterns were classified as cars (False Positives).
  - 3 cars were not identified (False Negatives).

Instance Is Detected \ Instance Exists	Yes	No
Yes	True Positive (TP)	False Positive (FP)
No	False Negative (FN)	True Negative (TN)

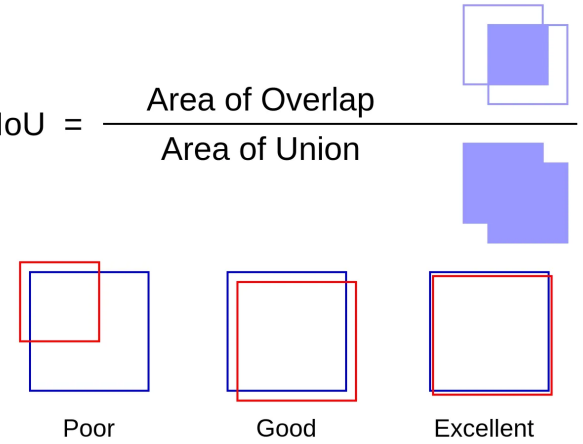
$$\text{Precision} : (7)/(7+2) = 77.8\%$$
$$\text{Recall} : 7/10 = 70\%$$

Measuring Quality

- ***F1 Score*: Classification Quality**
  - The F1 score is the *harmonic mean* of Precision and Recall: $F = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
  - It unifies Precision and Recall into one „goodness score“.

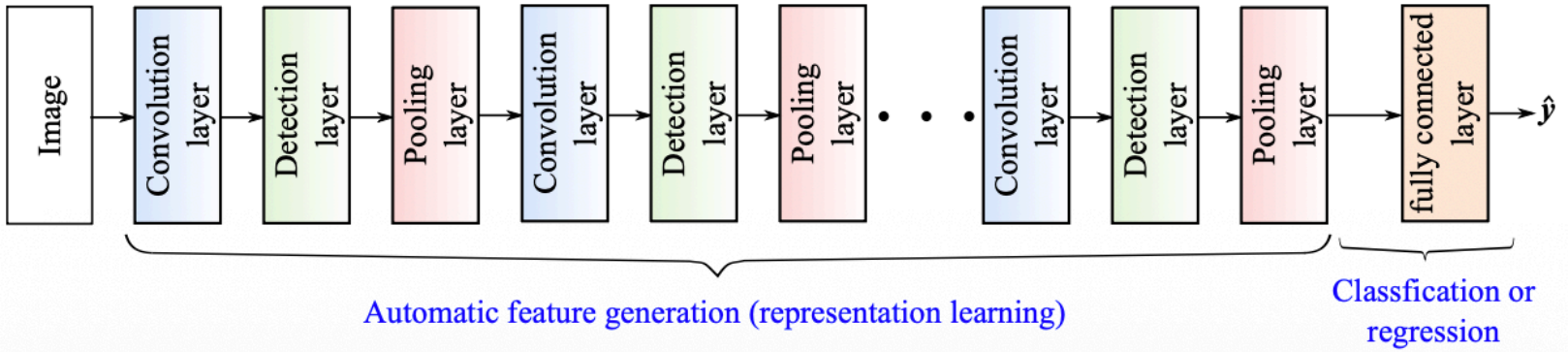
- **Intersection Over Union: Spatial Localization Quality**
  - Measures the normalized overlap of two bounding boxes (e.g. ground truth and prediction).
  - Gives a good estimate on precision of the bounding box extraction, vital for object **spatial localization** .

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Machine Learning: Convolutional Neural Networks

- **Convolutional Neural Networks (ger. Faltungsnetzwerke) are well suited for processing time series or image data, as they are able to find local features in the data and learn them.**
- **CNNs are very similar to multilayer perceptrons, but they require less parameters for the same number of hidden layers.**
- **This is realized by replacing at least one weighting operation $\mathbf{a}^\top \mathbf{x} + b$ by another linear operation: the Convolution or the Deterministic Cross-Correlation, respectively.**



- **Deep CNNs consist of a sequence of convolution, detection and pooling layers, completed by a fully connected layer.**

- **We can think of an image as a 2D function, $\mathrm{Img}(u,v) \rightarrow [0, I_{max}]$, where u, v are the image coordinates and Imax is the maximum intensity allowed by the image.**
- **Object characteristics take the form of local image changes in one or two directions.**
- **We can amplify these changes for later processing by deriving the image in both directions.**
- **The derivative again takes the form of an image.**

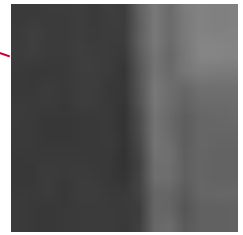




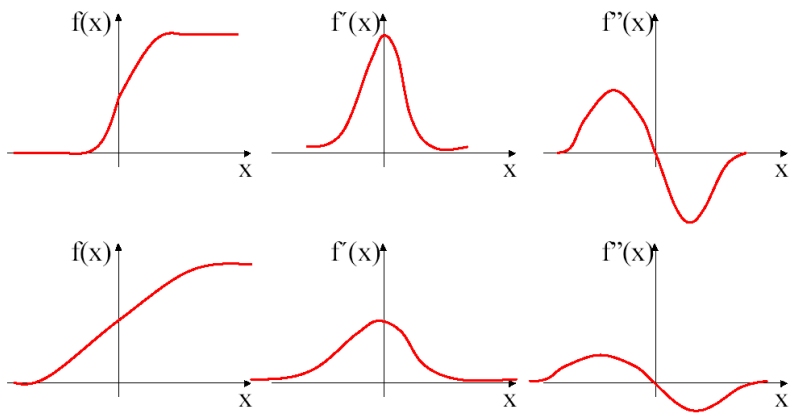
Low change



High change in two directions



High change in one direction



- **The convolution of two discrete D-dimensional signals $v(n_1, n_2, ..., n_D)$ and $w(n_1, n_2, ..., n_D)$ can be written as**

$$x(n_1, n_2, ..., n_D) = v(n_1, n_2, ..., n_D) *\overset{D}{\cdots}* w(n_1, n_2, ..., n_D)$$

$$= \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} \cdots \sum_{k_D=-\infty}^{\infty} v(n_1 + k_1, n_2 + k_2, \ldots, n_D + k_D) w(k_1, k_2, \ldots, k_D)$$

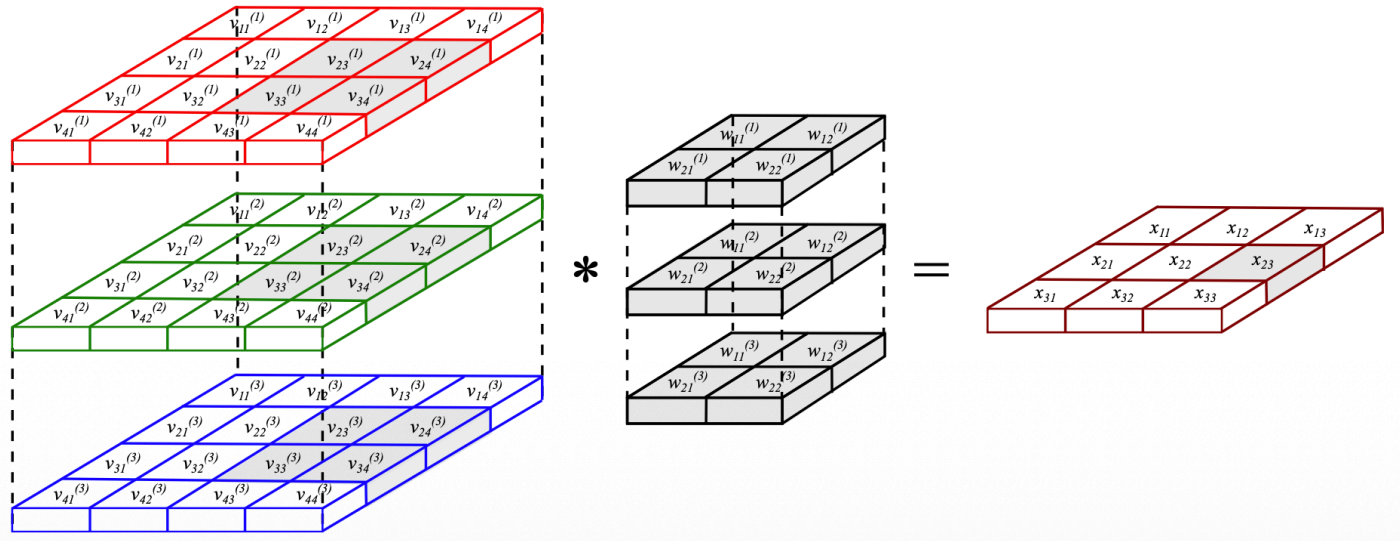
  (technically, that is the cross-correlation, convolution would be $v(n_1 - k_1, n_2 - k_2, \ldots)$, but in computer vision these are typically used interchangeably as they differ only by sign).

- **For D=2, the data structure can be interpreted as an image.**
- **We have seen convolution kernels in lecture 3.**

$$\frac{1}{16}\begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix} \qquad C_{\mathrm{Sx}} = \begin{vmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{vmatrix}, C_{\mathrm{Sy}} = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{vmatrix}$$

- **In CNNs, the kernel is *learned and not predetermined*.**

- **The advantage of the convolution operation is the capturing of local structure in an image, thereby learning the typical required patterns.**
- **In lecture 3 we have discussed that convolution is typically done on a grayscale image after converting RGB to grayscale.**
- **However we can think of RGB images as 3-dimensional data structures with the more general convolution equation.**



Convolution of 4x4x3 image with 2x2x3 kernel and adding up to a 3x3x1 image

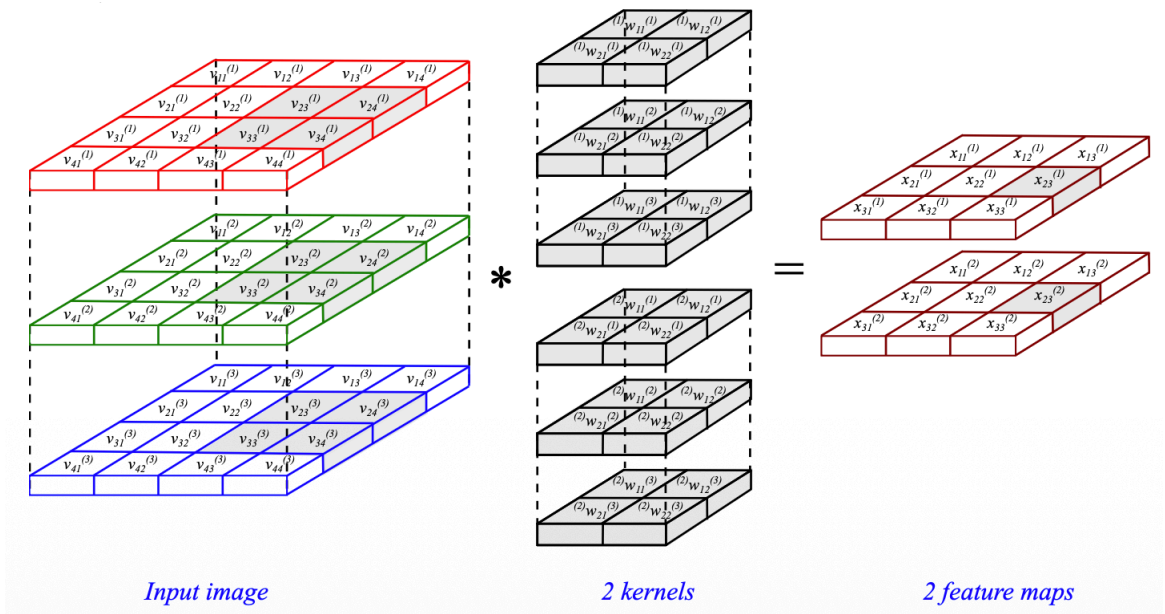
The values $x_{11}, \ldots, x_{33}$ are derived from the convolution of the 4x4x3 image with the 2x2x3 kernel:

$$\begin{aligned} x_{23} = & v_{23}^{(1)} w_{11}^{(1)} + v_{24}^{(1)} w_{12}^{(1)} + v_{33}^{(1)} w_{21}^{(1)} + v_{34}^{(1)} w_{22}^{(1)} \\ & + v_{23}^{(2)} w_{11}^{(2)} + v_{24}^{(2)} w_{12}^{(2)} + v_{33}^{(2)} w_{21}^{(2)} + v_{34}^{(2)} w_{22}^{(2)} \\ & + v_{23}^{(3)} w_{11}^{(3)} + v_{24}^{(3)} w_{12}^{(3)} + v_{33}^{(3)} w_{21}^{(3)} + v_{34}^{(3)} w_{22}^{(3)} \end{aligned}$$

where the high index denotes the R, G, and B channel.

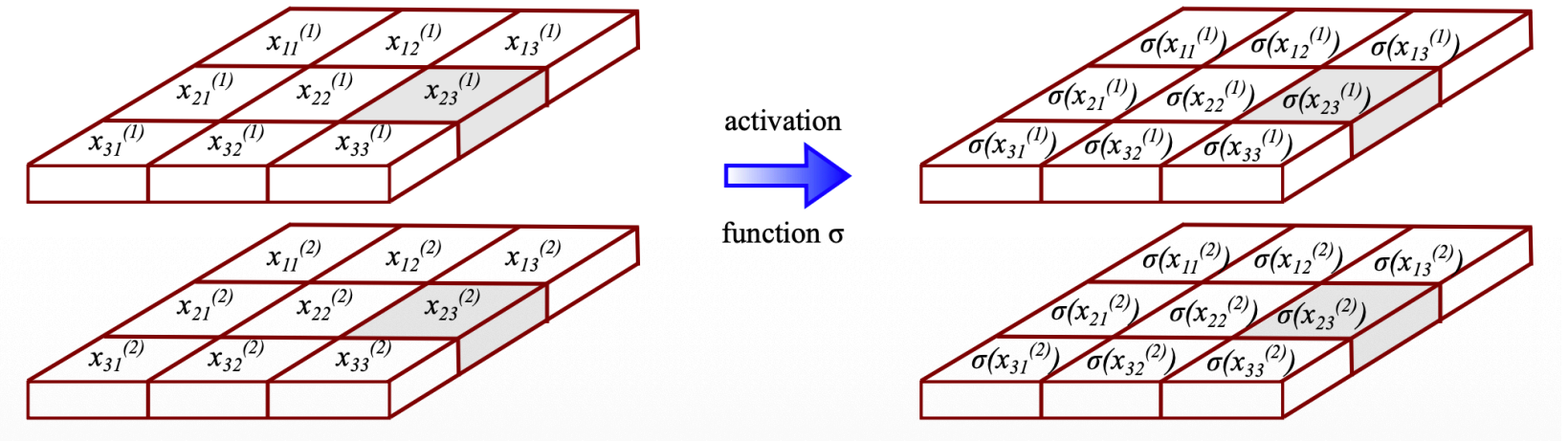
- **We can interpret the w_{ij} as the input weights to a neuron.**
- **In order to dertermine the 9 values $x_{11}, \ldots, x_{33}$ we only need to learn 3*4+1= 13 parameters.**
- **Solving the same task with a perceptron requires 16*3*9+9= 441 parameters.**

- **Usually we do not just generate one feature map from an image, but several.**
- **These are obtained by convolving with different kernels K_F .**

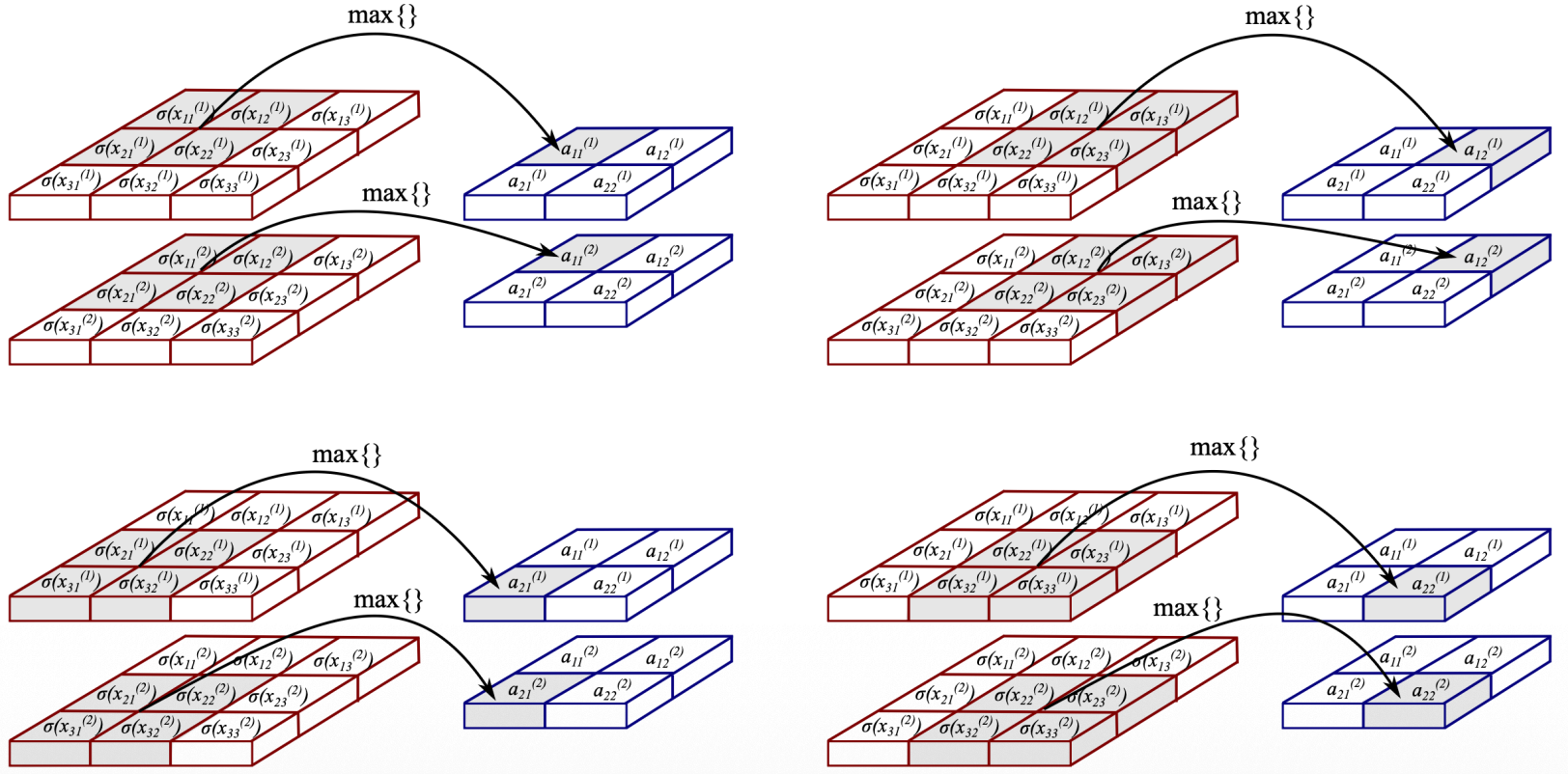


- **So far, we are in the convolution layer of a CNN, where everything is strictly linear.**

- **Detection Layers introduce nonlinearities to CNNs and are realized with activation functions, as previously discussed in the multilayer perceptron section.**
- **The activation functions affect the feature maps element-wise.**
- **Often the ReLu function, introduced in Subsection 2.1, is used as an activation function.**
- **The dimensions of the input and output keep the same size after the detection layer.**

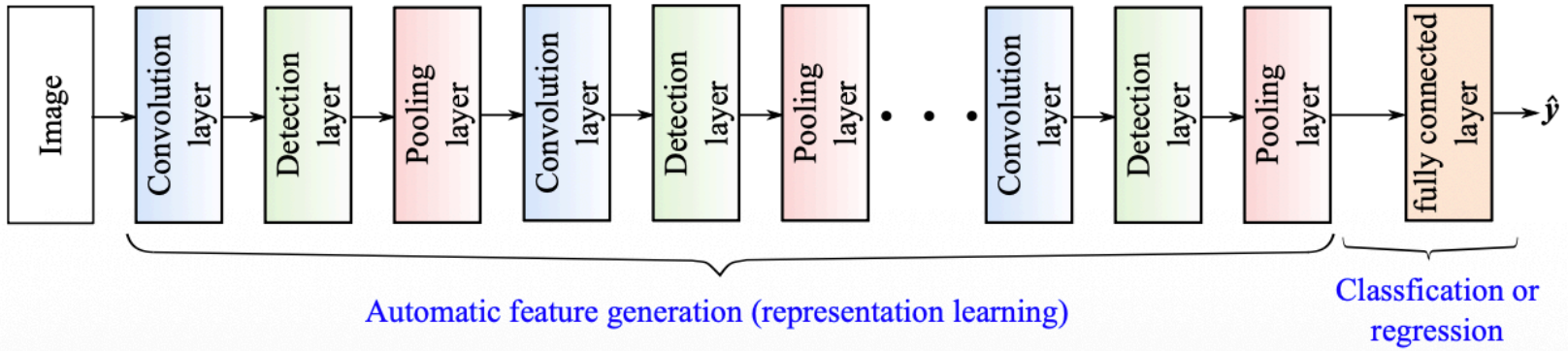


- **Pooling Layers, often also referred to as downsampling layers, reduce the resolution of the feature map by replacing areas of the feature map with statistical quantities.**
- **Pooling yields an invariance with regard to minor translations of the input, i. e., small local translations in the input data structure lead to no effect at the output of the pooling layer.**
- **Often „Max Pooling“ is used – only the maximum value from a section is kept and all the other values are discarded.**
- **Usually a quadratic region with length and width L_P is chosen, which is slid over the feature map with a defined step size (also named stride) S_P and the covered area is replaced by one statistical quantity, e. g., the maximum value.**

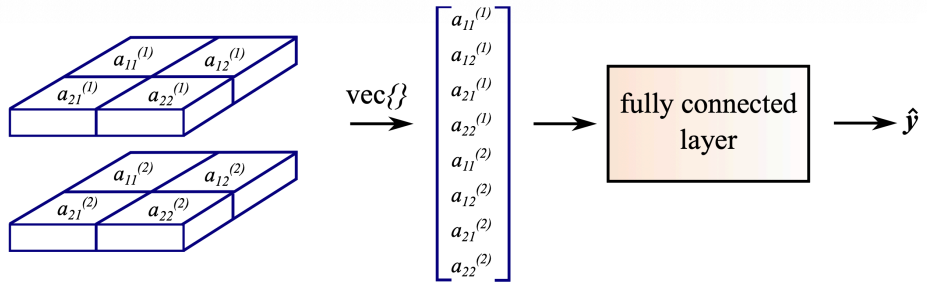


Max Pooling with $L_P = 2, S_P = 1$

- **A CNN usually is set up with several convolution, detection and pooling layers, finally followed by a fully connected layer.**



- **The fully connected layer transforms the final feature maps to the output of the network.**
- **Therefore the feature maps are rearranged to a single vector using the vec operator.**
- **This vector is then the input for the fully connected layer, which computes the output of the CNN.**



- **Backpropagation follows the same general rule for multilayer perceptrons, with the following specialties:**
  - The activation functions in the fully connected layer(s) and the detection layers are differentiated as usual.
  - The convolution layer does not have an activation function; its derivative is simply the image at the given position:

$$\frac{\partial x_{ij}}{\partial w_{kl}} = \frac{\partial}{\partial w_{kl}} \sum_m \sum_n v_{i+m,j+n} w_{mn} = v_{i+k,j+l}$$

  - The statistical function used in the pooling layers may not be differentiable. The maximum function for example is not. We can use a convention (a „subdifferential“):

$$\frac{\partial \max(p,q)}{\partial p} = \begin{cases} 1 & \text{if } p > q \\ 0 & \text{else} \end{cases}$$



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!